DEEP SURVEY · MIXTURE-OF-EXPERTS 2026-11-03

MoE 大模型深度调研
原理、架构演进、训练系统与推理部署

从 GShard 到 DeepSeek-V4.1 / Kimi K3 / GLM-5：
稀疏路由的数学、八次瓶颈迁移、EP 通信与 all-to-all、量化与分离式部署

1337	56	80	11
arXiv 检索候选	篇深度精读	篇速览	章正文

> MoE 的本质，是把「参数容量」与「每 token 计算量」解耦。
> 它省算力，但不省显存——这是所有推理工程难题的起点。

GShard · Switch · GLaM · ST-MoE · Mixtral · DeepSeekMoE · DeepSeek-V2 / V3 / V3.2 · DeepSeek-V4 / V4.1 · Qwen3 / Qwen3.8-Next · Kimi K2 / K3 · GLM-4.5 / GLM-5 · MiniMax-M2 · Step-3 · LongCat-Flash · Llama 4 · OLMoE · Nemotron-H

覆盖范围 1991 年自适应 Mixture of Experts → 2026 年 10 月最新技术报告
数据来源 arXiv 元数据与全文、官方技术报告、开源实现（DeepEP / EPLB / Tutel / MegaBlocks）
面向读者 推理引擎、CUDA 算子与系统工程师；假设读者熟悉 Transformer 与分布式训练基础

MoE SURVEY · 2026-11-03 56 deep reads · 80 quick scans · 1337 candidates

目 录

执行摘要 3

十条核心结论

1. 为什么 MoE 成为 2024-2026 的主流 4

1.1 稠密模型的"三堵墙" · 1.2 成本结构的量化对比 · 1.3 2026 年的市场格局

2. MoE 的数学基础：稀疏激活到底省了什么 5

2.1 从稠密 scaling 到条件计算 · 2.2 一层 MoE 的前向：公式与实现 · 2.3 容量因子与 token dropping · 2.4 负载均衡损失：为什么需要，以及它的代价 · 2.5 z-loss：稳定 router 的第二个旋钮 · 2.6 专家并行的通信量：一个必须记住的公式 · 2.7 稀疏度该取多少：scaling law 的答案

3. 路由与负载均衡：MoE 最活跃的战场 8

3.1 四代负载均衡方案的演进 · 3.2 aux-loss-free：把均衡从梯度里拿出来 · 3.3 Expert Choice：让专家选 token · 3.4 Soft MoE：用软分配消灭 drop · 3.5 其他路由范式 · 3.6 推理侧的负载均衡：EPLB · 3.7 一个容易被忽略的问题：路由坍缩的诊断

4. 架构演进：八个里程碑与"瓶颈迁移" 11

4.1 节点 1-2：从"统计分工"到"稀疏激活" · 4.2 节点 3-4：集群训练与开源产品化 · 4.3 节点 5：DeepSeekMoE 的"知识切分" · 4.4 节点 6：超稀疏扩容（V3 / K2） · 4.5 节点 7：动态计算（LongCat-Flash / Stepped MoE） · 4.6 节点 8：语义-物理解耦（V4 / K3） · 4.7 演进的三条主线

5. 模型代际全景：从 600B 到 2.8T 15

5.1 奠基期（2020-2024）：把 MoE 跑通 · 5.2 主力期（2025-2026）：稀疏度竞赛与系统协同 · 5.3 五个可读出的规律 · 5.4 三个值得单独说的模型

6. 训练系统：MoE 的稳定性、精度与流水线 20

6.1 稳定性：从 aux loss 到 aux-loss-free · 6.2 FP8 训练：DeepSeek-V3 的配方 · 6.3 DualPipe：把 all-to-all 藏进计算 · 6.4 Muon 与 MuonClip：优化器层面的稳定性 · 6.5 Upcycling：把稠密模型"长成"MoE · 6.6 RL 阶段：MoE 的新战场 · 6.7 训练成本：一个可复用的估算框架

7. 推理系统：MoE 真正的主战场 23

7.1 三个根本矛盾 · 7.2 并行策略：为什么 MoE 偏爱 EP · 7.3 all-to-all：MoE 的通信命脉 · 7.4 负载均衡：推理侧的三层机制 · 7.5 Kernel：grouped GEMM 与 fused MoE · 7.6 量化：MoE 的"显存救命稻草" · 7.7 Offloading：当显存装不下时 · 7.8 分离式部署（disaggregation）：2025-2026 的主线 · 7.9 PD 分离与 wide EP：V3 的生产部署 · 7.10 长上下文：KV cache 与 MoE 权重的显存战争 · 7.11 端侧与边缘：另一条战线 · 7.12 定量分析：MoE decode 的 roofline

8. 硬件协同：MoE 对芯片的要求 29

8.1 一张表看懂硬件瓶颈 · 8.2 算术强度的三档 · 8.3 NVLink 与 IB：两种通信的物理基础 · 8.4 计算-通信重叠的硬件基础 · 8.5 DeepSeek 的硬件建议（ISCA'25） · 8.6 一个被低估的指标：显存带宽利用率

9. 前沿方向：2026 年的五个战场 31

9.1 长上下文：注意力与 MoE 解耦 · 9.2 多模态 MoE · 9.3 MoE + RL：从"能训"到"训得稳" · 9.4 动态计算：让 token 决定预算 · 9.5 权重外置：把"死重"请出显存 · 9.6 理论：专家为什么会专业化 · 9.7 一个开放问题：专家到底学到了什么？

10. 产业落地：成本、部署与选型 34

10.1 训练成本对比 · 10.2 推理成本结构 · 10.3 部署案例：三种典型场景 · 10.4 选型建议：什么时候用 MoE · 10.5 一个务实的判断

11. 趋势判断与开放问题 36

11.1 十条判断 · 11.2 五个开放问题 · 11.3 给推理工程师的三条建议

附录 A · 术语表 38

附录 B · 56 篇核心论文深度卡 40

附录 C · 80 篇相关工作速览 55

执行摘要

本报告基于 **1337 篇** arXiv 候选论文的系统检索，深度精读 **56 篇**核心论文（含 2026 年 10 月最新技术报告），并对 **80 篇**相关工作进行速览。报告面向推理引擎与系统工程师，重点覆盖 MoE 的数学原理、架构演进、训练稳定性、推理系统与硬件协同。

十条核心结论

1. **MoE 的本质是"参数容量与每 token 计算量解耦"，它省算力但不省显存。** 671B 参数的 V3 每 token 只激活 37B，但全部 671B 权重必须驻留 HBM——这是所有推理工程难题的起点。
2. **稀疏度持续提升，激活参数保持稳定。** 从 GShard 的 2048 专家/top-2（约 4×），到 V3 的 256 专家/top-8（18×），再到 K3 的 896 专家/top-16（27×）、V4-Pro 的 384 专家/top-6（33×）。激活参数从 97B（GLaM）降到 37B（V3）、13B（V4-Flash）、6B（Qwen3.8-Next），但旗舰模型的激活参数仍维持在 30B 以上（多在 30–50B，K3 的 104B 是能力优先的例外）。
3. **架构演进是一部"瓶颈迁移史"。** 八个里程碑依次解决：输入空间划分（1991）→ 容量与 FLOPs 解耦（2017-2021）→ 集群训练通信（2020-2022）→ 开源产品化（2023）→ 知识切分（2024）→ 超稀疏扩容（2024-2025）→ 动态计算（2025）→ 语义与物理解耦（2026）。每一步都把瓶颈推到下一个位置。
4. **负载均衡经历了四代：aux loss → aux+z-loss → aux-loss-free → 推理侧 EPLB。** DeepSeek-V3 的 aux-loss-free（偏置项不进梯度）是 2024-2025 年最重要的训练技巧；推理侧的 EPLB（冗余专家 + 动态重排）是生产部署的必需。
5. **训练稳定性的三件套：aux-loss-free、FP8 细粒度量化、MuonClip。** V3 用 FP8 + aux-loss-free 做到 14.8T token 零不可恢复 spike；K2 用 MuonClip 做到 15.5T token 零 spike；Muon 正在取代 AdamW 成为大 MoE 的默认优化器。
6. **MoE decode 的瓶颈是 HBM 带宽，不是算力。** 算术强度 $AI = 2/b$（BF16 为 1 FLOP/byte），比 H100 的拐点（295 FLOP/byte）低近 300 倍。batch=1 时 GPU 算力利用率可能只有 1-2%；要进入 compute-bound，每专家 batch 需达到约 256。
7. **分离式部署（attention/FFN 分离）是 2025-2026 年推理系统的主线。** MegaScale-Infer、JANUS、Step-3 AFD、FinDEP 都把 attention 和 MoE FFN 放到不同的 GPU 池，各自独立扩缩容。JANUS 报告 4.7× 单卡吞吐提升。
8. **all-to-all 通信量 = $2Tkdb$，是 EP 的核心成本。** V3 单层约 939 MB、61 层约 57 GB。DeepEP（LL/HT 双模式 + hook 重叠）、NCCL EP（官方标准化 API）、UBEP（超节点优化）是当前三条主要路线。
9. **量化是 MoE 的"显存救命稻草"，且 MoE 比 dense 更鲁棒。** MoQE 证明专家权重可降到 2-bit；QMoE 把 1.6T 模型压到 160 GB（0.8 bit/param）；V4.1-Flash 用 CSA2 跨层复用 + FP4 KV cache，把每 token 的**全局** KV 占用压到 890 字节（约为 V4-Flash 的 1/4）。
10. **2026 年的旗舰格局：V4-Pro（1.6T/49B）、K3（2.8T/104B）、GLM-5（744B/40B）、Qwen3.8-Flash-Next（125B/6B）。** 共同趋势是"更大总参数 + 更小激活 + 更长上下文 + 混合注意力 + MoE 与注意力解耦"。

1. 为什么 MoE 成为 2024-2026 的主流

1.1 稠密模型的"三堵墙"

2020-2023 年，稠密模型的 scaling 遇到三堵墙：

1. **算力墙**：每 token 计算量随参数线性增长，推理成本无法承受；
2. **显存墙**：参数增长快于 HBM 容量增长；
3. **数据墙**：高质量 token 有限，大模型容易"训不足"。

MoE 解的是第一堵墙（算力），缓解第二堵墙（用稀疏激活降低"有效计算"对显存带宽的压力），但不解第三堵墙（反而需要更多数据）。

1.2 成本结构的量化对比

以"同等能力"为基准，比较 dense 与 MoE：

维度	Dense（假设 400B）	MoE（V3 671B/37B）	差异
训练 FLOPs	6 × 400B × 15T	6 × 37B × 14.8T	MoE ≈ 1/10
推理每 token FLOPs	2 × 400B	2 × 37B	MoE ≈ 1/10
推理显存	800 GB（BF16）	1.3 TB（BF16）	MoE ≈ 1.6×
推理通信	无	all-to-all 939 MB/层（61 层合计 57 GB）	MoE 新增
工程复杂度	低	高（EP/均衡/通信）	MoE 显著更高

结论：MoE 用"显存 + 工程复杂度"换"算力"。在算力贵、显存便宜、团队有系统能力的场景，MoE 划算；反之则不划算。

1.3 2026 年的市场格局

- **开源旗舰**：DeepSeek-V4、Kimi K3、GLM-5、Qwen3.8、MiniMax-M2 全部是 MoE；
- **闭源旗舰**：GPT-6、Gemini 4、Claude 4.6 的具体架构未公开，但业界普遍推测为 MoE 或混合架构；
- **端侧**：仍以 dense 小模型为主，MoE 在"端侧大模型"（如 Qwen3-30B-A3B）中有应用。

MoE 已经不是"一种选择"，而是"大模型的主流架构"。2026 年新发布的开源 100B+ 模型，几乎全部是 MoE。

2. MoE 的数学基础：稀疏激活到底省了什么

2.1 从稠密 scaling 到条件计算

稠密 Transformer 的成本结构可以用两个数字概括：

- **训练算力**：$C \approx 6ND$（N 为参数量，D 为 token 数）；
- **推理算力**：每 token 前向约 $2N$ FLOPs，且**每个参数都必须被读到**。

这意味着稠密模型扩大参数时，训练和推理成本同步线性上升。MoE 的核心主张是**把"参数容量"和"每 token 计算量"解耦**：把 Transformer 的 FFN 层复制成 E 个专家，每个 token 只路由到其中 k 个（$k \ll E$）。于是：

量	稠密模型	MoE
参数总量	N	$\approx E \cdot N_expert + N_shared$
每 token 激活参数	N	$\approx k \cdot N_expert + N_shared$
每 token FLOPs	$2N$	$\approx 2(k \cdot N_expert + N_shared)$
显存占用	$\propto N$	$\propto$ 总参数（全部专家都要驻留）

关键结论：MoE 省的是"计算"，不是"显存"。 这是理解 MoE 推理所有工程难题的起点——权重必须全量驻留 HBM，但每个 token 只用到其中一小部分。以 DeepSeek-V3 为例：671B 总参数、37B 激活参数，稀疏度约 18×；以 BF16 存储需约 1.3 TB 显存，而单 token 只读约 74 GB 权重。

2.2 一层 MoE 的前向：公式与实现

设输入 $x \in R^d$，门控矩阵 $W_g \in R^{d \times E}$。

(1) 门控打分

$$s = x \cdot W_g \in \mathbb{R}^E \tag{2-1}$$

路由打分

(2) Top-k 选择

$$\mathcal{T} = \mathrm{TopK}(s, k) \tag{2-2}$$

选中集合

(3) 门控权重（softmax 版，Switch/GShard 路线）

$$g_i = \exp(s_i) / \Sigma_{j \in \mathcal{T}} \exp(s_j) \quad (i \in \mathcal{T}); \quad g_i = 0 \quad (i \notin \mathcal{T}) \tag{2-3}$$

softmax 门控

(4) 加权求和

$$y = \Sigma_{i \in \mathcal{T}} g_i \cdot \mathrm{FFN}_i(x) \tag{2-4}$$

专家加权求和

DeepSeek 系列从 V2 起改用 **sigmoid 门控 + top-k 内归一化**：

$$g_i = \sigma(s_i) / \Sigma_{j \in \mathcal{T}} \sigma(s_j), \quad i \in \mathcal{T}$$ (2-5) sigmoid 门控

差异不只是换了个激活函数。softmax 让所有专家在同一个分母下**互相竞争**（一个专家分数高会压低其他所有专家），sigmoid 让每个专家**独立判断**"我是否适合这个 token"，再由 top-k 归一化保证权重和为 1。DeepSeek 的报告明确说这样"缓解了专家之间的竞争"，也让门控对专家数的扩展更稳定。2025 年的一篇分析（arXiv:2505.10860）进一步给出了统计解释：sigmoid 门控的梯度不会被其他专家的 logits 稀释，共享专家则承担了"通用知识"的容量，使路由专家更专注于细分模式。

2.3 容量因子与 token dropping

训练时每个专家的容量被固定为：

$$\text{capacity} = \text{CF} \times (T_{\text{batch}} \cdot k) / E$$ (2-6) 专家容量

- **CF > 1**：每个专家留余量，超出部分 padding，浪费算力；
- **CF < 1**：超出容量的 token 被**丢弃**（不经过该专家，残差直通），信息损失；
- **dropless**：不设上限，按实际负载动态分配（MegaBlocks 的块稀疏路线）。

Switch Transformer 训练用 CF=1.0~1.25，推理时 dropless；GShard 用 CF=1.25。**token dropping 是 MoE 训练不稳定和评测波动的常见来源**：同一句话在不同 batch 组成下可能被路由到不同专家、甚至被丢弃，导致输出不可复现。2025 年的 "Demons in the Detail"（arXiv:2501.11873）专门分析了负载均衡损失的实现细节，指出不同框架对 f_i 的计数口径（是否含 padding、是否按 top-k 重复计数）不同，会让"同样超参"训出完全不同的专家分布。

2.4 负载均衡损失：为什么需要，以及它的代价

路由是**自利**的：每个 token 独立选专家，模型会倾向于把 token 集中到少数"好用"的专家，形成**负载坍缩（load collapse）**。后果是过载专家丢 token、闲置专家浪费参数。

Switch Transformer 的辅助损失：

$$\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \Sigma_{i=1}^{E} f_i \cdot P_i$$ (2-7) 负载均衡损失

其中：

$$f_i = (1/T) \Sigma_{t=1}^{T} \mathbb{1}[i \in \text{TopK}(s_t, k)], \quad P_i = (1/T) \Sigma_{t=1}^{T} g_i(x_t)$$ (2-8) 负载占比与概率质量

- f_i 是**离散**的"分到专家 i 的 token 比例"（不可微，当作常数）；
- P_i 是**连续**的"路由概率均值"（可微，梯度从这里回传）；
- 乘 E 是为了让均匀分布时 $L_aux = \alpha$，与专家数无关。

这个设计有一个根本矛盾：**负载均衡是"batch 级"目标，语言建模是"token 级"目标**。α 调大，专家分布均匀但语言建模变差；α 调小，专家坍缩。而且 f_i 依赖当前 batch 的组成，batch 大小一变，最优 α 就变。

2.5 z-loss：稳定 router 的第二个旋钮

Router logits 过大会让 softmax 进入饱和区，梯度接近 0 或爆炸。ST-MoE 引入 **router z-loss**：

$$\mathcal{L}_z = (1/B)\,\Sigma_{b=1}^{B}\,(\log \Sigma_{j=1}^{E} \exp(s_j^{(b)}))^2 \qquad \text{(2-9) z-loss}$$

它惩罚 logits 的绝对尺度，不改变路由的相对排序。ST-MoE 用系数 0.001，报告称这是"训练 269B 参数 MoE 不崩"的关键之一。DeepSeek-V3 则用更直接的手段：门控 logits 用 FP32 计算、sigmoid 有界、再加 aux-loss-free 的偏置控制。

2.6 专家并行的通信量：一个必须记住的公式

当专家被切分到不同 GPU（Expert Parallelism, EP），每个 token 要"发"到它选中的专家所在设备，算完再"收"回来。单层、单 batch 的通信量：

$$V_{comm} = 2 \cdot T \cdot k \cdot d \cdot b \qquad \text{(2-10) all-to-all 通信量}$$

- T：batch 内 token 数；k：每 token 激活专家数；d：hidden 维度；b：每元素字节数；系数 2 = dispatch + combine。

代入 DeepSeek-V3 的量级（T=4096，k=8，d=7168，BF16 b=2）：

$$V = 2 \times 4096 \times 8 \times 7168 \times 2 \approx 939\ \text{MB / layer} \qquad \text{(2-11) V3 单层实算}$$

61 层就是约 **57 GB** 的跨设备流量——这还没算 all-to-all 的路由元数据、padding 和对端排队。这就是为什么 MoE 推理的瓶颈往往不在 GEMM，而在**通信**。

2.7 稀疏度该取多少：scaling law 的答案

Clark et al. (2022) 的 "Unified Scaling Laws for Routed Language Models" 给出了路由模型的 scaling 形式，核心结论是：**在固定算力下，最优的专家数/稀疏度随算力增长而增长**，但存在收益递减。工程上的经验区间：

- 2021-2022：稀疏度 4×~16×（GShard 600B 约 4×、GLaM 1.2T/97B 约 12×、Switch-C 1.6T/约 100B 激活约 16×）；
- 2023-2024：3.6×~18×（Mixtral 46.7B/12.9B 约 3.6×、DeepSeek-V3 671B/37B 约 18×）；
- 2025-2026：20×~33×（LongCat-Flash 560B/27B 约 20.7×、Kimi K3 2.8T/104B 约 27×、DeepSeek-V4-Pro 1.6T/49B 约 33×）。

稀疏度越高，同样算力下参数容量越大，但**通信占比、负载不均、专家利用率**三个问题同时恶化。这就是 2026 年研究的重心从"怎么更稀疏"转向"怎么把稀疏 MoE 跑得更快"的原因。

3. 路由与负载均衡：MoE 最活跃的战场

3.1 四代负载均衡方案的演进

代际	代表	机制	优点	代价
第一代	Switch / GShard	aux loss（token 比例 × 概率均值）	简单、可微	与 LM 目标冲突；batch 依赖
第二代	ST-MoE	aux loss + router z-loss	训练更稳	仍是超参敏感
第三代	DeepSeek-V3	**aux-loss-free**（偏置项，不进梯度）	不干扰 LM 梯度	需要监控偏置漂移
第四代	2026 系统侧	EPLB / 冗余专家 / 预测式调度	推理时动态均衡	副本占显存、重排有开销

3.2 aux-loss-free：把均衡从梯度里拿出来

DeepSeek-V3 的做法是给每个专家一个**偏置项** b_i，只用于 top-k 选择，不参与梯度：

$$s'_i = s_i + b_i, \quad \mathcal{T} = \mathrm{TopK}(s', k) \tag{3-1}$$

(3-1) 加偏置后选 top-k

每个训练 step 结束后按负载更新：

$$b_i \leftarrow b_i + \gamma \cdot \mathrm{sign}(\mathrm{load}_i - \overline{\mathrm{load}}) \tag{3-2}$$

(3-2) 偏置更新（不进梯度）

- 过载专家（$load_i > \overline{load}$）的偏置减小 → 下个 step 被选中概率降低；
- 欠载专家偏置增大 → 被选中概率提高；
- γ 是更新速率（V3 用 0.001），$sign$ 让更新幅度与负载偏差解耦。

为什么有效：均衡目标不再通过梯度回传，语言建模的梯度可以完全专注于"哪个专家更适合这个 token"。DeepSeek 报告称 14.8T token 训练全程无不可恢复的 loss spike、无回滚。后续分析（如 2025 年的 "Loss-Free Balancing" 复现）指出：偏置更新是**非平稳**的，如果 γ 太大，路由会在专家间振荡；如果太小，长尾负载无法纠正。实践中需要按层监控 b_i 的方差。

3.3 Expert Choice：让专家选 token

Zhou et al. (2022) 反过来：每个专家选 top-c 个 token，天然保证每个专家负载相同。

- **优点**：完美均衡、无 token drop（只要 c 够大）、负载与 batch 无关；
- **缺点**：**破坏了因果性**——一个 token 是否被某个专家处理，取决于同一 batch 里其他 token 的打分。这使自回归推理无法逐 token 进行，必须整批处理；也导致不同 batch 组成下输出不一致。

因此 Expert Choice 主要用于**训练/离线场景**，生产推理几乎不用。

3.4 Soft MoE：用软分配消灭 drop

Puigcerver et al. (2023) 的 Soft MoE 不做离散选择：每个专家接收所有 token 的**加权平均**（权重来自 softmax），输出再按权重加权回每个 token。

- **优点**：完全可微、无 token drop、负载天然均衡（每个专家处理固定数量的"槽位"）；
- **缺点**：计算量变成 $O(E \cdot slots \cdot d)$，不再随 k 稀疏；且每个 token 的输出混合了所有专家的结果，**丧失了"专家专业化"的稀疏性收益**。

Soft MoE 在大模型生产中很少见，但在多模态/小模型场景有应用。

3.5 其他路由范式

- **BASE Layers**（Lewis et al. 2021）：把 token 到专家的分配建模为**线性分配问题**，用匈牙利算法求最优均衡分配。理论上最均衡，但 $O(T^3)$ 的求解成本使其只适合小 batch。
- **Hash Layers**（Roller et al. 2021）：用固定哈希把 token 分到专家，**完全不需要路由参数**。负载天然均匀（哈希打散），但专家无法按内容专业化——本质是"随机稀疏"，效果通常不如学习式路由。
- **Sinkhorn / 最优传输**：用 Sinkhorn 迭代求软分配矩阵，兼顾均衡与可微，2023-2024 有多篇工作，但收敛速度和数值稳定性是工程难点。
- **动态专家数**：按 token 难度决定激活几个专家（easy token 用 1 个，hard token 用 4 个）。2025-2026 有多篇（如 Stepped MoE、ConceptMoE），核心问题是**难度估计器本身要训练**，且推理时动态分支对 GPU 不友好。

3.6 推理侧的负载均衡：EPLB

训练时的均衡目标是"每个专家处理差不多多的 token"；推理时的目标更复杂：

- **不同请求的专家偏好不同**（代码请求偏代码专家，数学请求偏推理专家）；
- **不同批次的专家热度随时间漂移**；
- **EP 组内不同 GPU 的负载可能差 2-3 倍**。

DeepSeek 开源的 **EPLB**（Expert Parallelism Load Balancer）做两件事：

1. **冗余专家**：把高负载专家复制多份，放到不同 GPU；
2. **层级重排**：先做"专家→GPU"的装箱（packing），再做"GPU→节点"的放置，减少跨节点 all-to-all。

2026 年的后续工作把这件事做得更细：FEPLB 用 copy engine 做"近零成本"的重排；CRAFT 做细粒度成本感知的专家复制；Least-Loaded EP 把"最少负载"作为路由的第二目标；NanoCP 处理变长负载下的 data-expert 并行解码不均衡。

3.7 一个容易被忽略的问题：路由坍缩的诊断

"专家坍缩"不是二值的（均衡/不均衡），而是连续谱。生产中常用的诊断指标：

- **专家负载熵**：$H = -\Sigma_i p_i \log p_i$，$p_i$ 是专家 i 的 token 占比。均匀时 $H = \log E$；
- **最大/最小负载比**：> 3 通常说明需要干预；
- **专家输出余弦相似度**：如果两个专家的输出高度相似，说明它们没有分化；

- **路由置信度分布**：top-1 与 top-2 的 logit 差。差值过大说明路由"过于自信"，容易过拟合到少数专家。

2025 年的 "Demons in the Detail" 给出一个反直觉结论：**过度均衡也会伤害性能**。当 aux loss 系数太大时，专家被迫处理不属于自己"专长"的 token，输出质量下降。均衡的目标不是"完全均匀"，而是"在语言建模损失不显著上升的前提下的最均匀"。

4. 架构演进：八个里程碑与"瓶颈迁移"

MoE 的历史不是"模型一代比一代大"的线性叙事，而是一部**瓶颈迁移史**：每一代架构解决一个核心矛盾，同时把瓶颈推到下一个位置。2026 年 8 月的一篇技术综述（arXiv:2608.08650）把这个过程整理成八个节点，每个节点标注"新设计变量 / 解决什么问题 / 把瓶颈推到哪里"。下表在此基础上补充了代表模型与关键数据。

#	里程碑	新设计变量	解决的核心问题	转移出的新瓶颈	代表
1	统计分工	gating + 局部专家	单一模型难以划分输入空间	所有专家都参与计算，容量绑定 FLOPs	Jacobs 1991, Jordan & Jacobs 1994
2	稀疏条件计算	top-k + capacity	总参数与激活 FLOPs 解耦	离散路由、坍缩、token drop	Shazeer 2017, GShard 2020, Switch 2021
3	集群级训练	EP + all-to-all + 稳定损失	Transformer MoE 能上集群	通信、straggler、数值/负载稳定性	GShard 600B / Switch 1.6T
4	开源产品化	decoder-only 闭环	经典 MoE 进入开放生态	大专家知识冗余、组合粒度粗	Mixtral 8×7B (2023)
5	知识切分	fine-grained + shared + device-limit	公共知识重复、专家组合扇出不足	小 GEMM、共享路径固定成本	DeepSeekMoE (2024), V2 (2024)
6	超稀疏扩容	更大 N、tiny experts、检索路由	固定激活预算下继续扩容量	加权 I/O、检索、低 token/专家	DeepSeek-V3 (2024), Kimi K2 (2025)
7	动态计算	adaptive-k / zero-compute experts	不同难度 token 用同一预算	控制开销、尾延迟、训练稳定性	LongCat-Flash (2025), Stepped MoE (2026)
8	语义-物理解耦	异构/跨层拓扑、结构化通信	语义路由被设备拓扑与通信绑死	调度复杂度、缓存一致性、大规模验证	DeepSeek-V4 (2026), Kimi K3 (2026)

> 来源：arXiv:2608.08650 的八节点表（Table 3）经本文核对与补充；代表模型与参数为各官方技术报告数据。

4.1 节点 1-2：从"统计分工"到"稀疏激活"

1991 年 Jacobs 等人的 "Adaptive Mixtures of Local Experts" 提出用门控网络把输入分配给不同子网络。这时所有专家都参与计算，MoE 只是**统计意义上的分工**（不同子网络拟合输入空间的不同区域），不是计算意义上的稀疏。

2017 年 Shazeer 等人的 Sparsely-Gated MoE 引入 top-k 稀疏门控与容量因子，第一次把"总参数"和"每 token 计算量"解耦。GShard（2020）把它推到 600B 参数、2048 个 TPU v3、4 天训练 100 语言翻译；Switch Transformer（2021）把 top-k 简化为 top-1，用 2048 专家做到 1.6T 参数，并首次证明 MoE 可以用 bfloat16 训练。

这一阶段的核心工程遗产是三个至今仍在用的机制：容量因子（capacity factor）、负载均衡辅助损失（aux loss）、专家并行（EP）。它们同时也是后续所有麻烦的源头。

4.2 节点 3-4：集群训练与开源产品化

GShard 的关键贡献不只是模型，而是**编译器层面的自动分片**：把 MoE 层表达成"可分片的计算图"，让 XLA 自动处理跨设备的 all-to-all。Switch 则证明 MoE 可以在 bfloat16 下稳定训练，并给出"选择性精度"（router 用 fp32、专家用 bf16）的配方。

2023 年 12 月 Mixtral 8×7B 是一个转折点：它用 8 个专家、top-2、47B 总参数/13B 激活，在开源社区证明了 decoder-only MoE 的产品化可行性。它的架构极其简单（就是 Mistral 7B 的 FFN 换成 8 个专家），但工程上第一次把"MoE 推理"变成普通开发者能跑的东西。

4.3 节点 5：DeepSeekMoE 的"知识切分"

DeepSeekMoE（2024-01）提出两个至今被广泛模仿的设计：

1. **细粒度专家（fine-grained expert segmentation）**：把每个专家切小，专家数从 8/16 增加到 64/160/256。这样同样激活参数下，token 可以由更多"小专家"组合，组合空间指数级增长；
2. **共享专家隔离（shared expert isolation）**：保留 1-2 个"总是激活"的共享专家，承载通用知识，让路由专家专注于细分模式。

DeepSeek-V2（2024-05）把这两点与 **MLA（Multi-head Latent Attention）** 结合：236B 总参数/21B 激活、128K 上下文、8.1T token。相比 DeepSeek 67B，训练成本降 42.5%、KV cache 减 93.3%、最大生成吞吐提升 5.76×。**MLA + MoE 的组合是 DeepSeek 系列成本优势的根基**：MLA 把 KV cache 压成 latent 向量，MoE 把 FFN 计算稀疏化，两者分别解决"长上下文显存"和"大模型算力"两个不同问题。

V2 还引入了 **device-limited routing**：每个 token 的候选专家限制在少数设备上，控制跨设备 fan-out。这是"语义路由受物理拓扑约束"的第一个显式设计，为节点 8 埋下伏笔。

4.4 节点 6：超稀疏扩容（V3 / K2）

DeepSeek-V3（2024-12）是这一阶段的集大成者：

- 671B 总参数、37B 激活（稀疏度约 18×）；
- 61 层、hidden 7168、MLA（128 heads × 128 dim，KV 压缩 512，query 压缩 1536）；
- 1 个共享专家 + 256 个路由专家，专家中间维 2048，每 token 激活 8 个路由专家；
- 每 token 最多发到 4 个节点（node-limited routing）；
- aux-loss-free 负载均衡 + MTP（多 token 预测，深度 1）；
- FP8 混合精度训练（fine-grained quantization）；
- 14.8T token，2.788M H800 GPU hours，全程无不可恢复 loss spike。

Kimi K2（2025-07）把稀疏度推到另一个量级：1.04T 总参数、32.6B 激活、384 个路由专家、top-8。它的关键创新是 **MuonClip**：在 Muon 优化器上加了 QK-clip，解决 attention logit 爆炸导致的训练不稳定，15.5T token 训练零 loss spike。

这一阶段的共同特征：专家数从几十增加到几百，专家宽度从 4096 降到 2048/1536，激活专家数稳定在 6-8。换句话说，**稀疏度提升靠"增加专家总数"，而不是"增加每 token 激活数"**。这带来两个直接后果：

单专家 GEMM 变小（效率下降）、专家负载更不均（通信和调度更难）。

4.5 节点 7：动态计算（LongCat-Flash / Stepped MoE）

LongCat-Flash（2025-09，美团）提出 **Zero-computation Experts**：一部分专家是"零计算"的（直接输出零或恒等），路由可以根据上下文动态决定激活多少专家。560B 总参数，每 token 激活 18.6B-31.3B（平均 27B），而不是固定值。

它的另一个设计是 **Shortcut-connected MoE**：把 MoE 层的输入直接连到输出（类似残差），扩大计算-通信重叠窗口。

动态计算的核心问题是**难度估计**：模型怎么知道这个 token 需要几个专家？LongCat 用路由分数的分布（top-k 与 top-(k+1) 的差距）作为代理。2026 年的 Stepped MoE、ConceptMoE 等工作进一步把"计算预算"变成可配置的超参。

4.6 节点 8：语义-物理解耦（V4 / K3）

2026 年的两个旗舰模型把 MoE 推到了新阶段：

DeepSeek-V4（2026-04）：

- V4-Pro：1.6T 总参数、49B 激活、61 层、hidden 7168、1 shared + 384 routed、top-6、专家中间维 3072；
- V4-Flash：284B 总参数、13B 激活、43 层、hidden 4096、1 shared + 256 routed、top-6、专家中间维 2048；
- 1M 上下文；CSA（Compressed Sparse Attention）+ HCA（Heavily Compressed Attention）混合注意力；
- mHC（Manifold-Constrained Hyper-Connections）改造残差；
- Muon 优化器；
- 门控从 Sigmoid 改为 **Sqrt(Softplus)**；
- 前 3 层 MoE 用 **Hash routing**（固定哈希，无路由参数）；
- 32T token。

Kimi K3（2026-07）：

- 2.8T 总参数、104B 激活、896 个路由专家、每 token 激活 16 个；
- KDA（Kimi Delta Attention）+ Attention Residuals；
- Stable LatentMoE；
- 1M 上下文、原生视觉；
- 相比 K2 的 scaling efficiency 提升约 2.5×。

这一阶段的关键词是"解耦"：

- **语义路由与物理执行解耦**：Hash routing、可学习的拓扑、跨层专家共享，让"哪个专家处理这个 token"不再被"专家在哪张卡上"绑死；
- **注意力与 FFN 解耦**：CSA/HCA 把长上下文注意力压到极低复杂度，FFN 的 MoE 独立扩展；
- **训练与推理解耦**：V4.1-Flash 的 CED 架构让 prefill 只激活 8B、decode 激活 16B，同一模型在不同阶段用不同计算量。

4.7 演进的三条主线

把八个节点压缩，可以看到三条同时推进的主线：

1. **容量线**：总参数从 600B → 1.6T → 2.8T，稀疏度从 4× → 18× → 27×；
2. **效率线**：每 token 激活参数从 97B（GLaM）→ 37B（V3）→ 13B（V4-Flash）→ 6B（Qwen3.8-Next），单位算力的能力持续提升；
3. **系统线**：从"单机 EP"到"跨节点 EP"到"分离式部署"到"语义-物理解耦"，每一步都是被通信和负载不均逼出来的。

一个反直觉的观察：2026 年的旗舰模型（V4-Pro 49B、K3 104B）激活参数比 2025 年的 V3（37B）更大，但单位激活参数的效率更高（V4 在 1M 上下文下单 token FLOPs 只有 V3.2 的 27%）。这说明"稀疏度"不再是唯一指标，**"有效计算密度"**（每 FLOP 产生的有效上下文/推理能力）成为新的竞争维度。

5. 模型代际全景：从 600B 到 2.8T

下表汇总 2020-2026 年主要 MoE 模型的公开参数。**所有数字以官方技术报告为准**；标注"官方博客"的条目来自厂商发布页而非 arXiv 论文；"未披露"表示报告未给出该项。

5.1 奠基期（2020-2024）：把 MoE 跑通

模型	时间	总参数	激活	路由专家	激活专家	层数	训练 token	上下文	注意力	关键设计
GShard	2020-06	600B	未披露	2048	top-2	—	—	—	—	首个大规模 Transformer MoE；2048 TPU v3 / 4 天
Switch-C	2021-01	1.6T	未披露	2048	top-1	32	—	—	—	top-1 简化；bf16 训练；选择性精度
GLaM	2021-12	1.2T	97B	64	top-2	32	1.6T	—	—	首个"激活参数"叙事
ST-MoE-32B	2022-02	269B	~32B 等效	32	top-2	—	—	—	—	router z-loss；训练稳定性设计指南
Mixtral 8×7B	2023-12	46.7B	12.9B	8	top-2	32	—	32K	GQA	开源产品化拐点
DeepSeekMoE-16B	2024-01	16.4B	2.8B	64+2	6+2	28	2T	4K	MHA	细粒度 + 共享专家
DeepSeek-V2	2024-05	236B	21B	160+2	6+2	60	8.1T	128K	MLA	device-limited routing；KV cache −93.3%
DBRX	2024-03	132B	36B	16	top-4	40	12T	32K	GQA	16 专家粗粒度
Grok-1	2024-03	314B	86B	8	top-2	64	—	8K	—	粗粒度、高激活比
Snowflake Arctic	2024-04	480B	17B	128	—	—	3.5T	—	—	极致稀疏（28×）
OLMoE	2024-09	6.9B	1.3B	64	top-8	16	5T	4K	MHA	全开源（权重/数据/日志）
Jamba	2024-03	52B	12B	16	top-2	—	—	256K	Mamba 混合	Transformer-Mamba 混合

模型	时间	总参数	激活	路由专家	激活专家	层数	训练 token	上下文	注意力	关键设计
Hunyuan-Large	2024-11	389B	52B	16	top-1	—	—	—	—	腾讯；top-1 粗粒度
Qwen1.5-MoE	2024-02	14.3B	2.7B	60+4	4+4	24	—	32K	—	细粒度 + 共享
Qwen2-57B-A14B	2024-06	57.3B	14B	64	top-8	28	—	128K	GQA	中等规模

5.2 主力期（2025-2026）：稀疏度竞赛与系统协同

模型	时间	总参数	激活	路由专家	激活专家	层数	训练 token	上下文	注意力	关键设计
DeepSeek-V3	2024-12	671B	37B	256+1	8+1	61	14.8T	128K	MLA	aux-loss-free、MTP、FP8、DualPipe
Qwen3-235B-A22B	2025-05	235B	22B	128	top-8	94	36T	128K	GQA	thinking/non-thinking 统一
Kimi K2	2025-07	1.04T	32.6B	384	top-8	61	15.5T	128K	MLA	MuonClip（QK-clip）；零 loss spike
GLM-4.5	2025-08	355B	32B	160	top-8	92	23T	128K	—	ARC 能力
GLM-5	2026-02	744B	40B	256	top-8	80	28.5T	200K	DSA	DSA 稀疏注意力；异步 RL
MiniMax-01	2025-01	456B	45.9B	32	top-2	64	—	4M	Lightning	线性注意力 + MoE
MiniMax-M2	2026-05	229.9B	9.8B	未披露	未披露	—	—	—	—	agentic 端到端；Forge RL
Step-3	2025-07	321B	38B	未披露	未披露	61	—	—	MFA	AFD（Attention-FFN 分离）

模型	时间	总参数	激活	路由专家	激活专家	层数	训练 token	上下文	注意力	关键设计
LongCat-Flash	2025-09	560B	27B 均值	—	动态	—	—	—	MLA	Zero-computation Experts；18.6-31.3B 动态
Hunyuan-TurboS	2025-05	389B	52B	16	top-1	—	—	—	Mamba 混合	Mamba-Transformer 协同
Llama 4 Scout	2025-04	109B	17B	16	top-1	48	官方博客	10M	iRoPE	early fusion；top-1
Llama 4 Maverick	2025-04	400B	17B	128	top-1	48	官方博客	1M	iRoPE	early fusion
Qwen3-Next-80B-A3B	2025-09	80B	3B	未披露	未披露	—	—	256K	GDN 混合	Gated DeltaNet + 全局注意力
Kimi Linear	2025-10	48B	3B	未披露	未披露	—	—	—	KDA+MLA	线性注意力首次全面超越全注意力
DeepSeek-V3.2	2025-12	671B（沿用 V3 主干）	37B	256+1	8+1	61	—	128K	DSA	DSA 稀疏注意力；RL 规模化
Kimi K3	2026-07	2.8T	104B	896	top-16	—	—	1M	KDA	Stable LatentMoE；原生视觉
DeepSeek-V4-Pro	2026-04	1.6T	49B	384+1	6+1	61	32T	1M	CSA/HCA	mHC、Muon、Hash routing（前 3 层）
DeepSeek-V4-Flash	2026-04	284B	13B	256+1	6+1	43	32T	1M	CSA/HCA	同上；1M 上下文 FLOPs 仅 V3.2 的 27%

模型	时间	总参数	激活	路由专家	激活专家	层数	训练 token	上下文	注意力	关键设计
DeepSeek-V4.1-Flash	2026-09	552B	16B decode / 8B prefill	—	—	—	45T	1M	CSA2	CED、FP4 KV（890 B/token）、SWA Bounded Replay
Qwen3.8-Flash-Next	2026-08	125B（+51B n-gram 外置）	6B	—	—	48	—	—	GDN+QSA	1/9 训练 FLOPs；Gated Residual
Nemotron 3.5 Super/Lightning	2026-10	—	—	—	—	—	—	—	Mamba 混合	开放高效 MoE

5.3 五个可读出的规律

(1) 激活参数不涨，总参数暴涨。 2024 年 V3 是 37B 激活；2026 年 V4-Pro 是 49B、K3 是 104B——激活参数缓慢上升，但总参数从 671B 涨到 1.6T/2.8T。**稀疏度（总/激活）从 18× 涨到 33×（V4-Pro）、27×（K3）。**

(2) 专家数持续增加，单专家变窄。 V3 的 256 专家、专家中间维 2048；V4-Pro 的 384 专家、中间维 3072；K3 的 896 专家。专家越窄，组合空间越大，但单专家 GEMM 越小，GPU 利用率越难做。

(3) 激活专家数稳定在 6-8（K3 是 16）。 这是工程折中：激活太多，通信量和计算量都上去；激活太少，路由容量不足。K3 敢用 16，是因为它的 896 专家足够细，单专家足够小。

(4) 注意力与 MoE 解耦。 2025 年后的旗舰模型几乎都用了"混合注意力"：MLA（DeepSeek）、DSA（V3.2/GLM-5）、CSA/HCA（V4）、KDA（K3/Kimi Linear）、GDN（Qwen3-Next）、Lightning（MiniMax）。MoE 负责 FFN 容量，注意力负责上下文效率，两者独立演进。

(5) 训练 token 量级跳变。 2024 年 14.8T（V3）、2025 年 15.5T（K2）/23T（GLM-4.5）/36T（Qwen3）、2026 年 28.5T（GLM-5）/32T（V4）/45T（V4.1-Flash）。**数据量已经超过" Chinchilla 最优"数倍**，说明 MoE 的容量需要海量数据才能填满——这也是为什么 MoE 训练越来越像"数据工程"而不是"模型工程"。

5.4 三个值得单独说的模型

DeepSeek-V3.2（2025-12）：它是"V3 主干 + DSA 稀疏注意力"的组合。DSA（DeepSeek Sparse Attention）用轻量 indexer 为每个 query 选 top-k 的 KV 块，把长上下文注意力的复杂度从 $O(L^2)$ 降到 $O(L \cdot k)$。V3.2 的意义在于：**它证明了 MoE 主干可以和稀疏注意力解耦升级**，不需要重新训练整个模型。

Kimi K3（2026-07）：2.8T 参数、896 专家、top-16，是当前公开的最大 MoE。它的 "Stable LatentMoE" 把专家路由放在低维 latent 空间做，减少路由本身的计算；KDA 则把线性注意力的表达力推到"公平对比下超过全注意力"。K3 的 2.5× scaling efficiency 提升，主要来自"专家更细 + 注意力更省 + 数据更好"三者叠加。

DeepSeek-V4.1-Flash（2026-09）：它不是"更大的模型"，而是"更省的模型"。552B 主干，prefill 只激活 8B、decode 激活 16B；CED 架构让 decoder 的 global KV 从 encoder 的 final hidden 投影而来；CSA2 做跨层 KV 复用；FP4 KV cache 把每 token 的全局 KV 压到 **890 字节**（约为 V4-Flash 的 1/4）。**这是"推理成本优先"的设计哲学**：模型能力靠 post-training 和 RL 提升，推理成本靠架构压缩。

6. 训练系统：MoE 的稳定性、精度与流水线

6.1 稳定性：从 aux loss 到 aux-loss-free

MoE 训练的第一大问题是**路由坍缩**：模型把 token 集中到少数专家，其余专家收不到梯度、逐渐退化。三代方案：

方案	机制	代表	副作用
aux loss	惩罚负载不均，梯度回传	Switch/GShard	干扰语言建模梯度；batch 依赖
aux + z-loss	再加 logits 尺度惩罚	ST-MoE	超参敏感；z-loss 系数需调
aux-loss-free	偏置项按负载更新，不进梯度	DeepSeek-V3	偏置漂移需监控

DeepSeek-V3 的 aux-loss-free 是 2024-2025 年最重要的训练技巧之一。它的更新规则极其简单：

$$b_i \leftarrow b_i + \gamma \cdot \mathrm{sign}(\mathrm{load}_i - \overline{\mathrm{load}}) \tag{3-2}$$

(3-2) 偏置更新（不进梯度）

但工程上有三个细节：

1. **偏置只加在 top-k 选择上，不加在门控权重上**。也就是说，b_i 改变"谁被选中"，但不改变"选中后权重多大"。这样语言建模的梯度完全不受均衡目标干扰。
2. **γ 要按层调**。浅层路由变化快，深层路由稳定；统一 γ 会让浅层振荡、深层不收敛。
3. **序列级均衡损失作为补充**。V3 后来加了 sequence-wise balance loss（只在序列维度统计），防止长序列内部的局部坍缩。V4 沿用并加强了这一点。

2026 年的后续研究（如 ReLibra、Router Drift Control）指出：**RL 阶段的路由漂移是新的不稳定源**。RL 会改变数据分布，导致训练时的专家负载分布与推理时不一致；解决方法是在 RL 时"重放"路由（routing replay），或者用轻量的负载预测器做在线校正。

6.2 FP8 训练：DeepSeek-V3 的配方

V3 是第一个在超大规模（671B）上验证 FP8 训练可行性的模型。它的配方可以概括为"**大部分算 FP8，关键处保高精度**"：

组件	精度	原因
GEMM（Fprop/Dgrad/Wgrad）	FP8（E4M3）	算力翻倍；Wgrad 的激活也用 FP8 存储，省显存
Embedding / Output Head	BF16	词表大，量化误差会累积
MoE Gating	FP32	路由对精度极敏感，FP8 会让 top-k 排序错乱
Normalization	BF16/FP32	数值范围大
Attention	BF16	softmax 对精度敏感
残差/累加	FP32	防止长链累加误差

Fine-grained quantization 是 V3 的核心技巧：

- 激活按 **1×128 tile** 分组，每组一个缩放因子；
- 权重按 **128×128 block** 分组，每组一个缩放因子；
- 这样比"整个张量一个 scale"精度高得多，又比"per-element scale"开销小。

V3 报告称 FP8 训练与 BF16 训练的 loss 曲线"几乎重合"，且训练速度提升约 1.5-1.8×（取决于 GEMM 占比）。

6.3 DualPipe：把 all-to-all 藏进计算

MoE 训练的通信量极大：跨节点 EP 的 all-to-all 让"计算:通信"降到约 **1:1**。V3 的 DualPipe 做了三件事：

1. **双向流水线**：从流水线两端同时注入 micro-batch，让前向和反向在时间上重叠；
2. **组件重排**：把一个 chunk 拆成 attention、dispatch、MLP、combine 四段，手动调整各段占用的 SM 比例，让 all-to-all 与计算并行；
3. **PP 通信与 EP 通信融合**：把流水线的 send/recv 和 all-to-all 的 dispatch/combine 放在同一个通信窗口，减少等待。

结果是 **all-to-all 和 PP 通信都被完全隐藏**，流水线气泡也减少。这套设计后来被 DeepEP、UniEP 等系统继承。

6.4 Muon 与 MuonClip：优化器层面的稳定性

Kimi K2 用 **MuonClip** 解决了一个具体问题：Muon 优化器的 token 效率高，但 attention 的 QK logits 容易爆炸（因为 Muon 对矩阵参数的更新幅度大）。MuonClip 的做法是在每次更新后对 Q/K 矩阵做 **QK-clip**：如果 $/QK^{\wedge}\to p/$ 超过阈值，就按比例缩小。K2 用这套配方训练 15.5T token，**零 loss spike**。

DeepSeek-V4 和 Qwen3.8-Next 也转向了 Muon。Qwen3.8 的报告说 Muon 让"最优学习率和 batch size 上移、batch-size warmup 不再必要、稳定性显著提升"。**Muon 正在取代 AdamW 成为大 MoE 训练的默认优化器**，这是 2025-2026 年一个容易被忽略但很重要的趋势。

6.5 Upcycling：把稠密模型"长成"MoE

Upcycling 的思路是：拿一个训练好的稠密模型，把它的 FFN 复制成多个专家，再继续训练。好处是省掉从零预训练的成本；坏处是**专家初始化完全相同**，需要额外的扰动/路由训练来分化。

2025 年的 scaling law 研究（arXiv:2502.03009）给出一个反直觉结论：upcycling 的收益不是单调的——**稠密模型和 upcycled MoE 之间存在一个"交互项"**：稠密模型越大，upcycling 的边际收益越小；数据量越大，upcycling 越划算。工程上的经验法则是：**upcycling 适合"中等稠密模型 + 大量新数据"的场景**，不适合"超大稠密模型 + 少量新数据"。

2026 年出现了多种 upcycling 变体：Expert Upcycling（按专家重要性选择性复制）、DivMoE（跨域专家组合）、STAPLE（静态稀疏 upcycling + embedding 模块）、Router Upcycling（复制 router 而非专家）。核心问题都是**如何让复制出来的专家尽快分化**。

6.6 RL 阶段：MoE 的新战场

2025 年之后，MoE 的训练重心从"预训练"转向"后训练 + RL"。三个代表性系统：

- **DeepSeek-R1**：GRPO（Group Relative Policy Optimization），用规则奖励做可验证任务的 RL。R1 证明了"纯 RL 可以激发推理能力"，但 R1 的 MoE 主干在 RL 中会出现路由漂移。
- **GLM-5**：异步 RL 基础设施，把"生成"和"训练"解耦到不同进程/设备，大幅提升 RL 吞吐；同时提出异步 agent RL 算法，支持长程交互。
- **MiniMax-M2**：Forge 系统，agent-native RL，windowed-FIFO 调度 + prefix-tree 合并 + 推理优化，支持白盒/黑盒 agent。

MoE + RL 的三个特殊问题：

1. **路由不一致**：训练时的路由（top-k 索引）和推理时的路由可能不同，导致 RL 的 importance ratio 失真。解法是 routing replay（把训练时的 top-k 索引存下来，推理时强制复用）。
2. **专家负载漂移**：RL 数据分布与预训练不同，某些专家会突然过载。解法是在线负载监控 + 动态冗余。
3. **长序列**：agentic RL 的轨迹动辄 100K+ token，KV cache 和 MoE 权重争抢 HBM。解法是 PD 分离 + KV 压缩（V4.1 的 FP4 KV 就是为此设计）。

6.7 训练成本：一个可复用的估算框架

DeepSeek-V3 的公开数字：**2.788M H800 GPU hours**，按当时 H800 租金约 *2/GPU-hour 估算，总成本约* **5.58M**。这个数字常被引用为"MoE 训练便宜"的证据，但需要注意口径：

- 它只算**预训练**，不含 SFT/RL/数据工程/失败重跑；
- 它假设**极高的 MFU**（V3 报告称约 40%），普通团队很难达到；
- 它依赖**极致的工程优化**（DualPipe、FP8、细粒度量化、通信重叠），这些优化本身就是数月的人力成本。

一个更现实的估算：**同等规模 MoE 的训练总成本 = GPU 租金 × (预训练小时 × 1.5~2.0) + 数据成本 + 人力成本**。1.5~2.0 的系数覆盖 SFT/RL/重跑/调试。

7. 推理系统：MoE 真正的主战场

7.1 三个根本矛盾

MoE 推理的困难可以归结为三个矛盾，所有系统工作都是在解这三个矛盾：

矛盾一：容量与带宽。 671B 参数（BF16 约 1.3 TB）必须全部驻留 HBM，但每个 token 只读 37B（约 74 GB）。这意味着 **99% 的权重在单 token 推理中是"死重"**——它们占着显存，却不产生计算。HBM 容量决定"能不能装下"，HBM 带宽决定"读得多快"。

矛盾二：稀疏与并行。 专家切分到多卡（EP）后，每个 token 要去不同卡上找专家，产生 all-to-all 通信。通信量与 token 数、激活专家数、hidden 维度成正比，且**随 EP 规模线性增长**。EP 越大，单卡负载越轻，但通信越贵。

矛盾三：均衡与效率。 真实请求的专家偏好高度不均（代码请求偏代码专家、数学请求偏推理专家）。EP 假设"每个专家负载差不多"，但现实中热门专家可能过载 3-5 倍，导致 straggler。均衡需要冗余专家/动态重排，但副本占显存、重排有开销。

7.2 并行策略：为什么 MoE 偏爱 EP

并行方式	切分对象	通信模式	适用
TP（张量并行）	单个矩阵的列/行	all-reduce（每层 2 次）	dense 层、attention；跨节点效率低
EP（专家并行）	不同专家到不同卡	all-to-all（dispatch/combine）	MoE FFN；跨节点可扩展
PP（流水线并行）	不同层到不同卡	p2p send/recv	超大模型；有气泡
DP（数据并行）	不同 batch 到不同卡	无（推理时）	attention 复制；提升吞吐
SP（序列并行）	序列维度切分	all-gather/reduce-scatter	长序列 attention

MoE 的核心选择是 **EP 而不是 TP**，原因：

1. **TP 的通信量与 hidden 维度成正比，且每层都要 all-reduce**；EP 只在 MoE 层做 all-to-all，且通信量与"被路由的 token 数"成正比——稀疏时更省。
2. **TP 要求每张卡都有完整的层输入**，无法利用"专家稀疏"；EP 让每张卡只放部分专家，天然匹配稀疏性。
3. **TP 跨节点效率低**（all-reduce 对延迟敏感）；EP 的 all-to-all 可以流水线化、可以与计算重叠。

但 EP 不是免费的：**EP 度越高，单专家 batch 越小，GEMM 效率越低**。V3 的 prefill 用 EP32（每专家 batch 足够大），decode 用 EP320（每卡一个专家，靠大 batch 摊薄）。这是一个典型的"按阶段选并行度"的设计。

7.3 all-to-all：MoE 的通信命脉

一次 MoE 层的前向包含两次 all-to-all：

1. **dispatch**：每个 token 的 hidden 向量发送到它选中的 k 个专家所在设备；
2. **combine**：专家算完后，结果按门控权重加权求和，送回原设备。

通信量（单层、单 batch）：

> $$V = 2 \cdot T \cdot k \cdot d \cdot b$$ (7-1) 单层 all-to-all 字节数

代入 V3（$T=4096$，$k=8$，$d=7168$，BF16 $b=2$）：单层约 **939 MB**，61 层约 **57 GB**。如果每层都同步等待，这个开销会吃掉大部分算力。所以所有高性能实现都做**通信-计算重叠**：

- **DeepEP**（DeepSeek 开源）：NVLink + RDMA 双通道；LL（low-latency）模式面向 decode 的小 batch，HT（high-throughput）模式面向 prefill/训练的大 batch；用 hook 机制把 dispatch/combine 拆成细粒度步骤，与 GEMM 交错执行。
- **NCCL EP**（NVIDIA，2026）：基于 NCCL Device API 的官方 EP 库，提供 `ncclEpDispatch` / `ncclEpCombine`；LL 模式针对 1-128 token 的 decode，HT 模式针对训练/prefill。它的意义是**把 EP 通信标准化**，让上层框架不必自己造轮子。
- **UBEP**（2026）：面向 NVL72/576 和 CloudMatrix384 这类"超节点"，解决 BSP 串行化、同步开销、距离无关调度导致的负载不均。
- **MegaScale-Infer 的 M2N 库**：消除 GPU-CPU 拷贝、组初始化开销和 GPU 同步，专门服务"attention-FFN 分离"后的 token 传输。

工程要点：all-to-all 的性能不只取决于带宽，还取决于**路由元数据的开销**。每个 token 要携带"目标专家 ID、原设备 ID、门控权重"，这些元数据的打包/解包可能比 hidden 向量本身还贵。DeepEP 的做法是把元数据压缩成 32-bit 的 token 索引，并在 GPU 上做 prefix-sum 排序。

7.4 负载均衡：推理侧的三层机制

训练侧的 aux-loss-free 解决"平均负载"，但推理侧的负载是**动态、偏斜、不可预测**的。生产系统通常有三层：

第一层：冗余专家（redundant experts）。 把高负载专家复制多份，放到不同 GPU。V3 的 prefill 部署有 32 个冗余专家（每 GPU 多放 1 个），decode 部署有 64 个 GPU 专门放冗余专家和共享专家。冗余专家的集合**每 10 分钟按在线统计更新**。

第二层：动态重排（rebalancing）。 在不增加跨节点通信的前提下，把专家在节点内重新排列，让每张卡的负载尽量均匀。V3 的 prefill 每 10 分钟重排一次；decode 因为"每卡一个专家"不需要重排。

第三层：请求级调度。 把"专家偏好相似"的请求放在同一 batch，减少跨节点 fan-out；或者用预测器预判下一层的专家激活，提前预取（PROBE）。2026 年的 LLEP 更进一步：**允许把过载设备的 token 和专家参数一起"搬"到空闲设备**，报告 5× 加速、4× 峰值显存下降。

一个反直觉的事实：2026 年的 LLEP 论文指出，**负载不均是"自然的、甚至有益的"**——领域专家集中处理领域 token，正是 MoE 想要的。强行均衡会破坏专业化。所以生产系统的目标不是"完全均匀"，而是"在 SLO 内接受不均衡"。

7.5 Kernel：grouped GEMM 与 fused MoE

MoE 的计算模式与 dense 完全不同：每个专家收到的 token 数不同（动态），需要 **grouped GEMM**（把多个不同大小的 GEMM 打包成一个 kernel 启动）。

三种实现路线：

1. **Token-major grouped GEMM**：按 token 分组，每个专家一个 tile。问题是当每专家 token 很少时，tile padding 严重、启动的 grid 很短命、带宽利用率低。
2. **Weight-major persistent kernel**（MonoMoE，2026）：把整个 decode step 的 token tile 放在 tensor-core 的 N 维度，CTA 按专家权重 tile 划分。一个 persistent grid 融合 routing、top-k、量化、两个 expert projection、激活、reduction。好处是**消除 expert-local token materialization、减少 padding**。
3. **Mega-kernel**（UniEP，2026）：把通信和计算融合进一个 kernel，用统一抽象适配不同 EP 优化策略（压缩、重叠、重排）。

融合（fusion）是另一个关键：gate + top-k + softmax + scatter + GEMM + weighted-sum 如果分开做，会有多次 kernel launch 和中间张量读写。vLLM 的 `fused_moe`、SGLang 的 `fused_experts`、TensorRT-LLM 的 MoE plugin 都在做这件事。MonoMoE 把这个思路推到极致：**整个 decode step 一个 kernel**。

7.6 量化：MoE 的"显存救命稻草"

MoE 对量化比 dense 模型更友好，原因有二：

1. **专家层比普通 FFN 更鲁棒**：MoQE（2023）发现，专家权重降到 2-bit，性能损失远小于同等 dense FFN；
2. **专家是"独立"的**：可以对不同专家用不同精度（热专家高精度、冷专家低精度），而不影响其他专家。

代表性工作：

工作	方法	效果
MoQE (2023)	只量化专家权重，2-bit	显存大降，性能损失小
QMoE (2023)	子 1-bit 压缩 + 定制解码 kernel	1.6T SwitchTransformer → <160 GB（0.8 bit/param，20×）
DynaExq (2025)	运行时混合精度分配（按专家热度）	单 GPU 硬显存约束下的在线优化
MonoMoE (2026)	block-wise 量化 + weight-major megakernel	decode 吞吐提升
V4.1-Flash (2026)	FP4 KV cache	890 B/token（V4-Flash 的 1/4）

工程要点：MoE 量化的难点不在"怎么量化"，而在"**量化后的 kernel 怎么写**"。低比特权重的解码需要专门的解包/反量化指令，且要和 grouped GEMM 融合。QMoE 的贡献一半是算法，一半是 bespoke GPU decoding kernel。

7.7 Offloading：当显存装不下时

单卡/少卡部署大 MoE 的标准做法是 **offloading**：把冷专家放在 CPU 内存/SSD，热专家留在 GPU，按需预取。

- **FineMoE**（2024）：从专家选择模式和 prompt 语义中提取"预取提示"，做细粒度 prefetch/cache/offload；
- **MoE-Lightning**（2024）：内存受限 GPU 上的高吞吐 MoE 推理，用分层 cache 和 pipeline；
- **KTransformers**（2025）：CPU-GPU 混合，把 MoE 专家放 CPU、attention 放 GPU，用 llama.cpp 风格的高效 CPU kernel；
- **VisMMOE**（2026）：视觉 token 剪枝不仅省计算，还**重塑专家需求**（visual-expert affinity），让专家访问更集中、更可预测，从而提升 offloading 命中率；
- **Specter**（2025）：量化 + 多步投机 offloading，把专家加载藏在投机解码的 draft 阶段。

offloading 的核心指标是"命中率"：如果热专家集合稳定，GPU 命中率高，延迟可控；如果请求分布漂移，命中率下降，延迟爆炸。所以生产系统通常把 offloading 作为"降级方案"，而不是首选。

7.8 分离式部署（disaggregation）：2025-2026 的主线

这是过去两年 MoE 推理系统最重要的架构变化。核心观察：**attention 和 FFN 的资源需求完全不同**。

- attention：prefill 时 compute-bound（大矩阵乘），decode 时 KV-cache-bound（读 KV）；
- MoE FFN：decode 时 memory-bound（读专家权重），prefill 时 compute-bound（大 GEMM）。

把它们放在同一组 GPU 上，只能取"最大公约数"的资源配置，浪费严重。于是出现 **attention-FFN 分离**：

系统	分离方式	关键机制
MegaScale-Infer (2024)	attention 和 FFN 分到不同 GPU 组	ping-pong pipeline；M2N 通信库
Step-3 AFD (2025)	Attention-FFN Disaggregation	专用子系统；MFA 注意力
JANUS (2025)	attention/MoE 分到不同 worker pool	两阶段通信；微秒级 activation scheduler；SLO-aware scaling；4.7× per-GPU 吞吐
FinDEP (2025)	DEP + 细粒度任务调度	支持 shared experts；任务重叠
AInfer-PD (2026)	prefill-decode 多路复用	通信安全的 in-place 复用

分离的代价是"token 要在两组 GPU 之间来回传"。所以分离系统必须解决：

1. **低延迟传输**：M2N、两阶段通信、零拷贝；
2. **流水线**：把 batch 切成 micro-batch，在 attention 组和 FFN 组之间"乒乓"；
3. **负载匹配**：attention 组和 FFN 组的 GPU 数量要按实际负载比例配置，否则一组等另一组。

7.9 PD 分离与 wide EP：V3 的生产部署

DeepSeek-V3 的公开部署方案是理解"大规模 MoE 推理"的最好样本：

Prefill 阶段：

- 最小部署单元：**4 节点 / 32 GPU**；
- attention：TP4 + SP，DP8；
- MoE：**EP32**（每 GPU 8 个专家）；
- **32 个冗余专家**（每 GPU 多放 1 个）；
- 同时处理两个 micro-batch，让一个的 attention/MoE 与另一个的 dispatch/combine 重叠；
- 每 10 分钟按在线统计更新冗余专家集合。

Decode 阶段：

- 最小部署单元：**40 节点 / 320 GPU**；
- attention：TP4 + SP，DP80；
- MoE：**EP320**（每 GPU 1 个专家）；
- **64 个 GPU 专门放冗余专家和共享专家**；
- all-to-all 用 IB 点对点 + **IBGDA**（GPU 直接发起 RDMA）；
- 每专家 batch 通常 < 256 token，**瓶颈是内存访问而非计算**；
- 只给 dispatch+MoE+combine 分配少量 SM，把大部分 SM 留给 attention。

为什么 decode 要 EP320？ 因为 decode 时每 token 只激活 8 个专家，如果 EP 度低，单卡要装很多专家、读很多权重；EP320 让每卡只装 1 个专家，权重读取量最小。代价是 all-to-all 的 fan-out 变大，所以用 IBGDA 和点对点传输压延迟。

wide EP 的边界：EP 度不是越高越好。当 EP 大到"每卡一个专家"后，再增加卡只会增加通信跳数，不会减少单卡权重读取。V3 的 EP320 基本是"每卡一专家"的极限；再往上需要"专家复制 + 请求路由"（EPLB）或"分离式部署"。

7.10 长上下文：KV cache 与 MoE 权重的显存战争

长上下文推理中，KV cache 和 MoE 权重争抢 HBM：

- 128K 上下文、BF16 KV、MLA 压缩后约 0.5-1 GB/请求；
- 671B MoE 权重约 1.3 TB；
- 一张 H100 80GB，装完权重后所剩无几。

解法是"**双向压缩**"：

1. **压 KV**：MLA（latent 压缩）、DSA（稀疏选择）、CSA/HCA（压缩 + 稀疏）、FP4 KV（V4.1，890 B/token）；
2. **压权重**：FP8/FP4 量化、专家 offloading、专家复制减少跨卡。

V4 的报告给出一个关键数字：**1M 上下文下，V4-Pro 的单 token 推理 FLOPs 只有 V3.2 的 27%，KV cache 只有 10%**。这不是靠"更大的模型"，而是靠 CSA/HCA 把注意力的复杂度从 $O(L^2)$ 压到接近 $O(L)$，再靠 mHC 和 Muon 保持训练稳定性。

7.11 端侧与边缘：另一条战线

MoE 在端侧有天然优势（激活参数小、算力需求低），但劣势是**权重总量大、显存不够**。2026 年的几条路线：

- **CPU-GPU 混合**（清华，2026）：SLP 流式加载 prefill，双 RTX 5090 上 1800 token/s、45K prompt 30 秒内完成；AVX-512 FP8 GEMV 让 CPU 原生跑 FP8；本地部署达到云级 SLO。
- **Apple Silicon**：多节点 EP 在 Mac Studio 集群上跑私有 LLM；Apple NPU 上的 MoE 推理。
- **3D-MoE**（2026）：3D 堆叠芯片上的 MoE decode 硬件-软件协同设计。
- **CoMoE**（2026）： commodity GPU 上的民主化 MoE 推理。

端侧的核心矛盾是"**权重总量 vs 显存带宽**"：即使只激活 3B 参数，也要从 100B+ 权重中挑出这 3B。所以端侧 MoE 的瓶颈几乎总是"读权重"，而不是"算"。

7.12 定量分析：MoE decode 的 roofline

用 roofline 模型可以精确解释"为什么 MoE decode 是 memory-bound"。

单 token、单层的算术强度（arithmetic intensity）：

$$\mathrm{AI} = (2 \cdot \mathrm{W_{act}}) / (\mathrm{W_{act}} \cdot \mathrm{b}) = 2 / \mathrm{b} \quad \mathrm{FLOP/byte}$$

(7-2) MoE decode 算术强度

- BF16（$b=2$）：$AI = 1$ FLOP/byte；
- FP8（$b=1$）：$AI = 2$；
- FP4（$b=0.5$）：$AI = 4$。

H100 SXM 的 BF16 峰值约 989 TFLOPS，HBM 带宽 3.35 TB/s，**拐点（ridge point）约 295 FLOP/byte**。MoE decode 的 $AI=1$，**比拐点低近 300 倍**——完全 memory-bound。

batch 的作用：如果一次处理 B 个 token，权重只读一次，$AI = 2B/b$。要让 BF16 达到拐点，需要 $B \approx 295$。这解释了为什么 V3 decode 要把"每专家 batch"做到 256 左右：**低于这个数，GPU 算力大量闲置；高于这个数，才进入 compute-bound 区间**。

这也解释了为什么"小 batch 的 MoE 推理"是灾难：batch=1 时，GPU 的算力利用率可能只有 1-2%，绝大部分时间在等 HBM。所有"低延迟 MoE 推理"的系统工作（DeepEP LL 模式、MonoMoE、投机解码）本质上都在解决"如何在小 batch 下仍把带宽用满"。

8. 硬件协同：MoE 对芯片的要求

8.1 一张表看懂硬件瓶颈

指标	H100 SXM	H200	B200	GB200 NVL72（单 GPU 等效）
HBM 容量	80 GB	141 GB	192 GB	192 GB
HBM 带宽	3.35 TB/s	4.8 TB/s	8 TB/s	8 TB/s
BF16 峰值	989 TFLOPS	989 TFLOPS	~2250 TFLOPS	~2250 TFLOPS
FP8 峰值	1979 TFLOPS	1979 TFLOPS	~4500 TFLOPS	~4500 TFLOPS
NVLink 带宽	900 GB/s	900 GB/s	1.8 TB/s	1.8 TB/s（域内）
节点间	IB 400 Gb/s	IB 400 Gb/s	IB 800 Gb/s	NVLink 域内 72 GPU

MoE 的核心矛盾在这张表里一目了然：

- 671B BF16 权重需要 1.3 TB → 需要 17 张 H100 或 7 张 B200 才能装下（还不算 KV）；
- 但 decode 时每 token 只读 37B → 单卡带宽利用率极低；
- 所以必须"多卡装权重 + EP 并行 + 大 batch"三件事一起做。

8.2 算术强度的三档

精度	每参数字节	decode AI（batch=1）	H100 拐点	差距
BF16	2	1 FLOP/byte	295	295×
FP8	1	2 FLOP/byte	590（FP8 峰值/带宽）	295×
FP4	0.5	4 FLOP/byte	1180	295×

结论：无论用什么精度，单 batch decode 都是 memory-bound，差距稳定在约 300 倍。精度提升（FP8/FP4）只是把"分子"和"分母"同比例放大，**不改变 memory-bound 的本质**。真正改变本质的是 batch size：$AI = 2B/b$，batch 上去才进入 compute-bound。

8.3 NVLink 与 IB：两种通信的物理基础

- **NVLink**：节点内 GPU 互联，H100 900 GB/s、B200 1.8 TB/s。all-to-all 的节点内部分走 NVLink。
- **IB / RoCE**：节点间互联，400/800 Gb/s per port。跨节点 all-to-all 走 IB。
- **IBGDA**：GPU 直接发起 RDMA，绕过 CPU，降低延迟。V3 的 decode 用它。
- **NVLink SHARP / 交换机内归约**：把 all-reduce 的部分计算下放到交换机，减少 GPU 工作量。

拓扑决定性能：EP 的 all-to-all 是"多对多"模式，对拓扑敏感。胖树（fat-tree）拓扑下，跨节点跳数少；环状/3D torus 拓扑下，跳数多。V3 的 node-limited routing（每 token 最多 4 个节点）就是为了控制跳数。

8.4 计算-通信重叠的硬件基础

MoE 的高性能实现依赖三个硬件特性：

1. **异步拷贝引擎（Copy Engine）**：NVLink 的 copy engine 可以在不占用 SM 的情况下搬数据。FEPLB（2026）利用这一点做"近零成本"的专家重排——重排的数据搬运完全在后台进行，不抢计算资源。
2. **TMA（Tensor Memory Accelerator）**：Hopper 起，TMA 可以在全局内存和共享内存之间做异步块拷贝，减少地址计算开销。MoE 的 grouped GEMM 用它加载专家权重。
3. **分布式共享内存（DSMEM）**：Hopper 的 thread block cluster 允许同一 cluster 内的 SM 直接访问彼此的共享内存。DeepEP 用它做节点内 all-to-all 的低延迟路径。

8.5 DeepSeek 的硬件建议（ISCA'25）

DeepSeek 在 ISCA'25 的论文（arXiv:2505.09343）里直接给芯片设计提了建议，核心有三条：

1. **HBM 容量比带宽更重要**：MoE 的瓶颈是"装不下"，不是"读不快"（虽然两者都重要）。他们建议未来芯片的 HBM 容量增长速度要快于带宽。
2. **互联要支持"多对多"模式**：MoE 的 all-to-all 不是规整的 all-reduce，需要更灵活的互联拓扑和更低的延迟。
3. **计算单元要支持"小 GEMM 高效"**：MoE 的单专家 GEMM 很小（batch 可能只有几十），需要硬件对小矩阵乘更友好。

这三条在 2026 年的硬件里部分实现了：B200 的 192GB HBM、NVLink 域内 72 GPU、SM100 的 FP4 支持，都是在往这个方向走。

8.6 一个被低估的指标：显存带宽利用率

MoE decode 的性能可以用一个简单公式估算：

$$\text{吞吐} \approx (\text{HBM 带宽} \times \text{带宽利用率}) / \text{每 token 读取字节数} \tag{8-1}$$

(8-1) 带宽受限下的吞吐上界

以 H100（3.35 TB/s）、BF16、每 token 读 74 GB（37B 激活参数）为例：

$$\text{吞吐} \approx (3.35 \times 10^{12} \times 0.7) / (74 \times 10^{9}) \approx 32 \text{ token/s} \tag{8-2}$$

(8-2) 以 V3 decode 为例

这与实测的 30-40 token/s 吻合。**要提高吞吐，只有三条路**：提高带宽利用率（更好的 kernel）、减少每 token 读取字节（量化/更稀疏）、增加 batch（摊薄权重读取）。

9. 前沿方向：2026 年的五个战场

9.1 长上下文：注意力与 MoE 解耦

2025 年之后，长上下文不再是"把 RoPE 拉长"，而是"重新设计注意力"。四条路线：

路线 A：稀疏选择（DSA / QSA）。 DeepSeek Sparse Attention（V3.2、GLM-5）用一个轻量 indexer 为每个 query 选 top-k 的 KV 块，只对选中的块做注意力。Qwen Sparse Attention（Qwen3.8-Next）类似，但用"微块粒度"打分。核心是 **indexer 要足够便宜**——它的计算量必须远小于主注意力。

路线 B：压缩 + 稀疏（CSA/HCA）。 DeepSeek-V4 的 CSA 先把序列压缩 m 倍（$m=4$），再在压缩后的 KV 上做稀疏选择（top-512/1024）；HCA 用更激进的压缩（$m'=128$）。两者交替使用：CSA 保细节，HCA 保长程。V4 报告称 1M 上下文下 FLOPs 只有 V3.2 的 27%。

路线 C：线性注意力（KDA / GDN / Lightning）。 Kimi Delta Attention（K3、Kimi Linear）用"对角+低秩"转移矩阵做线性注意力，Kimi Linear 报告"公平对比下超过全注意力"。Gated DeltaNet（Qwen3-Next）用门控 Delta 规则。MiniMax-01 的 Lightning Attention 把线性注意力做到 4M 上下文。

路线 D：混合（Hybrid）。 Qwen3-Next 用"每 4 层一个全注意力 + 其余 GDN"；Kimi Linear 用"KDA + MLA"层间混合；Jamba/Nemotron-H 用"Mamba + Transformer + MoE"混合。混合的目的是**用少量全注意力层保表达力，用大量线性层省成本**。

MoE 在其中的角色：注意力负责"上下文效率"，MoE 负责"参数容量"，两者独立扩展。V4 的 CSA/HCA 与 MoE 是正交的；K3 的 KDA 与 896 专家也是正交的。**"注意力-MoE 解耦"是 2026 年架构设计的默认范式**。

9.2 多模态 MoE

多模态 MoE 的核心问题是"**模态间的专家分配**"：

- **DeepSeek-VL2**：动态 tiling 视觉编码 + DeepSeekMoE + MLA，处理任意长宽比的高分辨率图像；
- **Qwen3-Omni**：全模态（文本/图像/音频/视频），MoE 主干；
- **Kimi K3**：原生视觉，896 专家；
- **DeepSeek-V4.1-Flash**：多模态，CED 架构，prefill 8B / decode 16B；
- **VisMMOE**（2026）：视觉 token 剪枝改善 offloading，利用 visual-expert affinity。

关键发现：视觉 token 数量远大于文本 token（一张高分辨率图可能几千 token），会**放大专家负载不均**。所以多模态 MoE 需要"模态感知的负载均衡"——V4.1 明确引入了 modality-specific load balancing（图像和文本 token 分开均衡）。

9.3 MoE + RL：从"能训"到"训得稳"

2025-2026 年，RL 成为 MoE 后训练的核心。三个系统性问题：

1. **路由一致性**：训练时 top-k 索引 vs 推理时 top-k 索引不一致 → importance ratio 失真。解法：routing replay（存下训练时的索引）。
2. **负载漂移**：RL 数据分布变化 → 专家负载突变。解法：在线监控 + 动态冗余 + ReLibra 的"路由重放引导均衡"。

3. **长轨迹**：agentic RL 的轨迹 100K+ token，KV cache 与 MoE 权重争 HBM。解法：PD 分离 + KV 压缩 + 异步 RL（GLM-5 的 generation/training 解耦）。

Router Drift Control（2026）给出一个诊断指标：训练时与推理时的 top-k 索引重合率。低于 80% 就需要干预。

9.4 动态计算：让 token 决定预算

固定 top-k 的问题是"简单 token 和困难 token 用同样的计算量"。动态计算让预算随 token 变化：

- **LongCat-Flash**（2025）：zero-computation experts，激活 18.6-31.3B（平均 27B）；
- **Stepped MoE**（2026）：segment-level routing，可配置推理复杂度；
- **ConceptMoE**（2026）：multi-ratio，一个模型多个计算预算；
- **Training-Free Halving**（2026）：推理时把激活专家数减半，无需重训。

工程难点：动态分支对 GPU 不友好（不同 token 走不同路径 → warp divergence、负载不均）。所以生产系统通常用"分段"（segment）而不是"逐 token"动态。

9.5 权重外置：把"死重"请出显存

MoE 的 99% 权重在单 token 推理中是"死重"。2026 年出现"权重外置"思路：

- **Qwen3.8-Flash-Next**：51B 参数的 n-gram embedding 表放在 host memory，按需 prefetch；
- **DeepSeek-V4.1-Flash**：Engram（稀疏访问的条件记忆），把部分记忆放 SSD/host；
- **KTransformers**：专家放 CPU，attention 放 GPU。

核心权衡：外置省显存，但引入 PCIe/SSD 带宽瓶颈。适合"显存受限 + 请求稀疏"的场景，不适合"高并发 + 低延迟"。

9.6 理论：专家为什么会专业化

2022 年的理论工作（arXiv:2208.02813）证明：**MoE 的成功依赖两个条件**——(1) 数据有内在的簇结构；(2) 专家是非线性的。Router 学到的是"簇中心特征"，把复杂问题分解成多个线性子问题，每个专家解决一个。

这个理论解释了为什么： - 专家数不能无限增加（簇的数量有限）； - 专家必须非线性（线性专家无法拟合簇内结构）； - 路由坍缩是"簇合并"的表现（数据簇不够分明时，多个专家退化成同一个）。

2025 年的统计学习分析（arXiv:2505.10860）进一步证明：**共享专家提高样本效率**（因为它承担了跨簇的通用成分），**normalized sigmoid 门控比 softmax 更适合细粒度专家**（因为 softmax 的全局竞争会压制小专家）。

9.7 一个开放问题：专家到底学到了什么？

2024-2026 年有多篇"专家可解释性"工作，结论并不一致：

- OLMoE 的分析显示路由有"高专业化"（不同专家处理不同领域）；
- 但也有工作发现"专家冗余"（多个专家输出高度相似）；
- DeepSeek-V3 的报告说"专家确实分化，但分化程度低于预期"。

目前的共识：专家专业化是"统计意义上的"，不是"语义意义上的"。一个专家可能同时处理"Python 代码"和"法律条文"，因为它们在表示空间里接近。**不要指望 MoE 的专家像"部门"一样分工**。

10. 产业落地：成本、部署与选型

10.1 训练成本对比

模型	训练 GPU hours	硬件	估算成本	备注
DeepSeek-V3	2.788M H800	2048 H800	≈ $5.58M	仅预训练；MFU ≈ 40%
DeepSeek-V4	未披露	未披露	—	32T token；规模约为 V3 的 2×
Kimi K2	未披露	未披露	—	15.5T token；MuonClip
Llama 4 Maverick	未披露	未披露	—	官方博客称"创纪录规模"

注意口径：公开的 GPU hours 通常只算"预训练的有效计算"，不含： - 数据清洗/合成（可能占 30-50% 人力）； - SFT/RL（可能再占 20-50% 算力）； - 失败重跑/调参（1.5-2× 系数）； - 推理部署的工程成本。

10.2 推理成本结构

MoE 推理成本可以拆成四块：

成本项	占比（典型）	决定因素
权重显存占用	40-60%	总参数 × 精度
KV cache	20-40%	上下文长度 × 并发 × KV 压缩率
通信（all-to-all）	10-30%	EP 度 × token 数 × 互联带宽
计算（GEMM）	10-20%	激活参数 × batch × 精度

MoE 的成本优势来自"计算便宜"（激活参数小），但**成本劣势来自"显存贵"**（总参数大）。所以 MoE 的经济性取决于"**显存/算力价格比**"：如果显存便宜、算力贵（如 GPU 云），MoE 划算；如果显存贵、算力便宜，dense 可能更划算。

10.3 部署案例：三种典型场景

场景 A：云端高并发 API（DeepSeek-V3 模式）

- PD 分离：prefill 32 GPU（EP32），decode 320 GPU（EP320）；
- 冗余专家 + 每 10 分钟重排；
- IBGDA 低延迟 all-to-all；
- 适合：吞吐优先、延迟容忍度高（P99 几百 ms）。

场景 B：企业私有化部署（单机/少机）

- EP + offloading：热专家在 GPU，冷专家在 CPU/SSD；
- 量化：FP8/INT4 专家权重；
- 适合：显存受限、请求量中等、延迟要求 < 1s。

场景 C：端侧/边缘（手机/PC）

- CPU-GPU 混合：attention 在 GPU，专家在 CPU；
- 小 batch + 投机解码；
- 适合：离线、隐私敏感、延迟要求宽松。

10.4 选型建议：什么时候用 MoE

适合 MoE 的场景：

1. **训练算力有限、但推理算力充足**：MoE 用更少训练 FLOPs 达到同等能力；
2. **需要大容量知识、但单次推理预算有限**：MoE 的"大容量 + 小激活"正好匹配；
3. **有大规模 GPU 集群、能接受复杂工程**：EP/PD 分离/通信优化需要团队能力；
4. **请求模式可预测、专家负载稳定**：负载均衡的成本可控。

不适合 MoE 的场景：

1. **显存极度受限**（单卡 24GB 以下）：总参数装不下，offloading 延迟不可接受；
2. **超低延迟**（< 100ms）：all-to-all 的延迟难以压到极致；
3. **小团队、无系统优化能力**：MoE 的工程复杂度远高于 dense；
4. **请求分布高度漂移**：专家负载频繁失衡，冗余专家和重排跟不上。

10.5 一个务实的判断

2026 年的现实是：**MoE 是"有系统能力的团队"的架构，不是"所有团队"的架构**。DeepSeek/Kimi/GLM 能用 MoE，是因为他们有： - 自己的训练框架（DualPipe、MuonClip）； - 自己的推理引擎（DeepEP、EPLB）； - 自己的集群（IBGPD、NVLink 域）。

普通团队用开源 MoE 模型（Qwen3、Mixtral、OLMoE）做推理，瓶颈通常在"显存不够"和"通信太慢"，而不是"模型不好"。所以选型时，**先算显存和带宽，再算能力**。

11. 趋势判断与开放问题

11.1 十条判断

1. **稀疏度还会涨，但增速放缓。** 从 18×（V3）到 27×（K3）到 33×（V4-Pro），下一步可能是 40-50×，但受限于"单专家 GEMM 太小"和"负载不均"。
2. **激活参数会稳定在 10-50B 区间。** 太小则容量不足，太大则推理成本高。K3 的 104B 是"能力优先"的例外。
3. **注意力与 MoE 彻底解耦。** 2026 年的旗舰模型没有一个是"纯全注意力 + MoE"；CSA/HCA、DSA、KDA、GDN 成为标配。
4. **分离式部署成为云端默认。** attention 和 FFN 的资源需求不同，分离是必然；下一步是"分离 + 动态扩缩容"。
5. **量化从"可选"变"必需"。** FP8 训练 + FP4/INT4 推理是 2026 年的标准组合；MoE 对量化的鲁棒性使其成为"量化最友好的架构"。
6. **RL 成为 MoE 后训练的主战场。** 预训练差距缩小后，RL 的数据/算法/系统成为差异化来源；异步 RL、agentic RL 是重点。
7. **动态计算从"研究"走向"产品"。** LongCat 的 zero-computation experts 是第一个产品级动态 MoE；下一步是"按请求难度动态分配 GPU"。
8. **权重外置（offloading/Engram）会在端侧普及。** 显存受限场景下，"热权重在 GPU、冷权重在 CPU/SSD"是唯一可行路径。
9. **MoE 的可解释性仍是开放问题。** 专家专业化是统计性的，不是语义性的；"专家即部门"的直觉是错的。
10. **MoE 不会取代 dense，而是分化。** 端侧/小模型用 dense（简单、省显存），云端/大模型用 MoE（容量大、算力省）；两者长期共存。

11.2 五个开放问题

1. **最优稀疏度的理论边界是什么？** 目前的 scaling law 只给出经验拟合，没有"为什么"的解释。
2. **专家专业化能否被显式控制？** 能否设计"可解释的专家分工"（如"代码专家""数学专家"），而不是让路由自己学？
3. **动态计算的难度估计器怎么训？** 目前的代理指标（路由分数差）不可靠，需要更好的"难度信号"。
4. **MoE 的 RL 稳定性有没有理论保证？** routing replay 是工程手段，缺乏理论分析。
5. **MoE 与稀疏注意力的最优组合是什么？** CSA/HCA、DSA、KDA 各有优劣，缺乏统一的比较框架。

11.3 给推理工程师的三条建议

1. **先把 roofline 算清楚。** MoE decode 的瓶颈几乎总是 HBM 带宽，不是算力。优化顺序：量化 → 减少每 token 读取字节 → 增大 batch → 通信重叠。
2. **不要迷信"更大的 EP"。** EP 度超过"每卡一专家"后，收益递减、通信成本上升。先算"每专家 batch"是否够大，再决定 EP 度。

3. **负载均衡是持续运营，不是一次性配置。** 生产环境要监控专家负载熵、最大/最小负载比、路由置信度，并按小时/天级更新冗余专家。

附录 A · 术语表

缩写/术语	全称	中文解释
MoE	Mixture-of-Experts	混合专家：把 FFN 拆成多个"专家"，每个 token 只激活其中一部分
Expert	Expert Network	专家：通常是一个独立的 FFN 子网络
Router / Gate	Gating Network	路由/门控：决定每个 token 送给哪些专家的线性层
Top-k	Top-k Routing	每个 token 选分数最高的 k 个专家
Capacity Factor (CF)	Capacity Factor	容量因子：每个专家最多处理的 token 数上限的倍率
Token Dropping	Token Dropping	专家容量满后，多余 token 被丢弃（不经过该专家）
Aux Loss	Auxiliary Loss	辅助损失：惩罚专家负载不均的额外损失项
Z-loss	Router Z-loss	惩罚 router logits 尺度过大的损失项，稳定训练
Aux-loss-free	Auxiliary-Loss-Free Balancing	用偏置项（不进梯度）做负载均衡，DeepSeek-V3 提出
Shared Expert	Shared Expert	总是激活的专家，承载通用知识
Fine-grained Expert	Fine-grained Expert Segmentation	把专家切小、数量增多，增加组合空间
EP	Expert Parallelism	专家并行：把不同专家放到不同 GPU
All-to-all	All-to-all Communication	多对多通信：每个设备向所有设备发送不同数据
Dispatch / Combine	Dispatch / Combine	MoE 的两次 all-to-all：发送 token 到专家 / 收回结果
TP / PP / DP / SP	Tensor / Pipeline / Data / Sequence Parallelism	张量/流水线/数据/序列并行
DeepEP	Deep Expert Parallelism	DeepSeek 开源的 EP 通信库（LL/HT 双模式）
EPLB	Expert Parallelism Load Balancer	DeepSeek 开源的推理侧专家负载均衡器
Grouped GEMM	Grouped GEMM	把多个不同大小的 GEMM 打包成一个 kernel

缩写/术语	全称	中文解释
Fused MoE	Fused MoE Kernel	把 gate/top-k/GEMM/加权求和融合成一个 kernel
PD 分离	Prefill-Decode Disaggregation	把预填充和解码放到不同 GPU 组
AFD	Attention-FFN Disaggregation	把注意力和 FFN 放到不同 GPU 组（Step-3）
KV Cache	Key-Value Cache	自回归推理缓存的历史 K/V
MLA	Multi-head Latent Attention	多头潜在注意力：把 KV 压成 latent 向量（DeepSeek）
DSA	DeepSeek Sparse Attention	用轻量 indexer 选 top-k KV 块的稀疏注意力
CSA / HCA	Compressed Sparse Attention / Heavily Compressed Attention	V4 的两种压缩注意力，交替使用
KDA	Kimi Delta Attention	Kimi 的线性注意力（对角+低秩转移矩阵）
GDN	Gated DeltaNet	门控 Delta 规则线性注意力（Qwen3-Next）
Upcycling	Sparse Upcycling	把稠密模型复制成专家，继续训练成 MoE
Muon / MuonClip	Muon Optimizer / Muon with QK-clip	矩阵级优化器；K2 加 QK-clip 防 logit 爆炸
Routing Replay	Routing Replay	存下训练时的 top-k 索引，推理时复用，保证一致性
Roofline	Roofline Model	用算术强度与硬件拐点判断 compute/memory bound 的模型
Arithmetic Intensity	Arithmetic Intensity	算术强度：每字节内存访问对应的 FLOPs
MFU	Model FLOPs Utilization	模型算力利用率：实际 FLOPs / 峰值 FLOPs
IBGDA	InfiniBand GPU Direct Async	GPU 直接发起 RDMA，绕过 CPU
TTFT / TPOT	Time To First Token / Time Per Output Token	首 token 延迟 / 每输出 token 时间
SLO	Service Level Objective	服务等级目标（延迟/吞吐承诺）

附录 B · 56 篇核心论文深度卡

按主题分组，每张卡给出「一句话结论 / 核心问题 / 方法 / 数据与实验 / 局限」。卡片顺序即阅读顺序：先综述建立框架，再看奠基工作、训练系统、模型代际、推理系统、量化、多模态与理论。

综述与总览（3 篇）

#01 综述与总览 2024 · 多机构（CUHK/SJTU 等） ★ 8.0

A Survey on Mixture of Experts in Large Language Models

arXiv:2407.06204 · arxiv.org/abs/2407.06204

MoE 领域最系统的综述，提出门控/专家/路由/系统四层分类学。

核心问题	把 MoE 拆成 gating、expert、routing、system 四个设计维度，逐一比较已有工作。
方法	① 给出稀疏 vs 稠密激活的对比框架；② 汇总 20+ 辅助损失函数与系数配置；③ 整理开源实现与超参表。
数据/实验	覆盖 2024 年前主要 MoE 模型；Table 2/3 给出参数、专家数、hidden 维的横向对比。
局限	偏算法侧，对 2025-2026 的推理系统（DeepEP/EPLB/分离部署）覆盖不足。

#02 综述与总览 2026 · 综述（Codex 辅助） ★ 9.0

The Evolution of Mixture-of-Experts Architectures in LLMs: Routing, Topology, Balance, EP

arXiv:2608.08650 · arxiv.org/abs/2608.08650

用'八里程碑 + 四控制平面'重构 MoE 演进史，是本报告架构章节的主要框架来源。

核心问题	MoE 的历史是瓶颈迁移史：每个里程碑解决一个矛盾，同时把瓶颈推到下一个位置。
方法	① 八节点依赖图（6 主线 + 2 正交分支）；② 四控制平面（拓扑/路由/均衡/EP）；③ 等预算预训练实验。
数据/实验	明确 DeepSeek-V2 236B/21B、Kimi K2 1.04T/32.6B/384 专家/top-8；给出 CV_expert 负载变异系数定义。
局限	综述性质，缺少一手实验；部分模型数据需与官方报告交叉核对。

#03 综述与总览 2024 · CUHK/SJTU ★ 8.0

A Survey on Inference Optimization Techniques for Mixture of Experts Models

arXiv:2412.14219 · arxiv.org/abs/2412.14219

MoE 推理优化的全景综述：计算、显存、通信、调度四条线。

核心问题	MoE 推理的瓶颈是'显存容量 + 带宽 + 通信'三者耦合，优化必须系统级联合设计。
方法	① 按'计算/存储/通信/调度'分类优化技术；② 汇总量化、offloading、EP、kernel 融合；③ 给出评测基准。
数据/实验	覆盖 2024 年前的推理系统（DeepSpeed-MoE、Tutel、FasterMoE、MoE-Infinity 等）。
局限	对 2025-2026 的分离式部署（JANUS/Step-3 AFD）未覆盖。

奠基与路由机制（9 篇）

#04 奠基与路由机制 2020 · Google ★ 9.0

GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding

arXiv:2006.16668 · arxiv.org/abs/2006.16668

第一个把 Transformer MoE 推到 600B 并跑在 2048 TPU 上的工作，确立 EP + 容量因子 + aux loss 三件套。

核心问题	用轻量注解 API + XLA 自动分片，让 MoE 的跨设备计算对用户透明。
方法	① top-2 门控 + 容量因子；② 自动分片（XLA）；③ 随机路由 + aux loss 均衡。
数据/实验	600B 参数；2048 TPU v3；4 天训练 100 语言翻译；sub-linear 计算成本。
局限	随机路由牺牲了学习式路由的表达力；容量因子导致 token drop。

#05 奠基与路由机制 2021 · Google ★ 9.0

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

arXiv:2101.03961 · arxiv.org/abs/2101.03961

把 top-k 简化为 top-1，用 2048 专家做到 1.6T，并首次证明 MoE 可用 bf16 训练。

核心问题	路由越简单越好：top-1 降低通信与计算，配合选择性精度和初始化即可稳定训练。
方法	① top-1 路由；② 选择性精度（router fp32、专家 bf16）；③ 专家初始化缩放 + 更高专家 dropout。
数据/实验	1.6T 参数（Switch-C）；2048 专家；相同算力下预训练加速最高 7×；101 语言全覆盖。
局限	top-1 的表达力弱于 top-2；1.6T 模型实际难以部署。

#06 奠基与路由机制 2022 · Google ★ 8.0

ST-MoE: Designing Stable and Transferable Sparse Expert Models

arXiv:2202.08906 · arxiv.org/abs/2202.08906

系统研究 MoE 训练不稳定性，提出 router z-loss，把稀疏模型做到 269B 并首次在迁移学习上 SOTA。

核心问题	MoE 的不稳定性主要来自 router logits 尺度失控；z-loss 是最有效的稳定器。
方法	① router z-loss（系数 0.001）；② 选择性精度；③ 专家 dropout + 微调策略。
数据/实验	269B 参数，计算成本相当于 32B dense；SuperGLUE/ARC/XSum/WebQA 全面 SOTA。
局限	z-loss 系数需按模型规模调；对推理系统无涉及。

#07 奠基与路由机制 2022 · Google ★ 8.0

Mixture-of-Experts with Expert Choice Routing

arXiv:2202.09368 · arxiv.org/abs/2202.09368

让专家选 token 而非 token 选专家，天然均衡，训练收敛快 2×。

核心问题	负载均衡不应靠损失函数，而应靠'分配方向'的反转：专家选 token 保证每个专家负载相同。
方法	① 专家选 top-c token；② 可变激活专家数；③ 与 top-1/top-2 同算力对比。
数据/实验	8B/64E 模型收敛速度 >2×；11 个 GLUE/SuperGLUE 任务提升。
局限	破坏因果性，无法逐 token 自回归推理；不同 batch 组成输出不一致。

#08 奠基与路由机制 2023 • Google DeepMind ★ 8.0

From Sparse to Soft Mixtures of Experts

arXiv:2308.00951 • arxiv.org/abs/2308.00951

> 用软分配（所有 token 加权混合后送给每个专家）消灭 token drop 和路由不稳定。

核心问题	离散路由是万恶之源；软分配让 MoE 完全可微，负载天然均衡。
方法	① 每个专家接收所有 token 的加权平均；② 槽位（slot）数决定计算量；③ 输出按权重加权回 token。
数据/实验	Soft MoE Huge/14：128 专家/16 层，参数比 ViT-H 多 40×，推理时间仅 +2%。
局限	计算量不再稀疏（与专家数成正比）；丧失专家专业化的稀疏收益；大 LLM 少用。

#09 奠基与路由机制 2021 • Meta AI ★ 7.0

BASE Layers: Simplifying Training of Large, Sparse Models

arXiv:2103.16716 • arxiv.org/abs/2103.16716

> 把 token-专家分配建模为线性分配问题，用匈牙利算法求最优均衡。

核心问题	均衡路由可以是一个组合优化问题，而不是启发式或辅助损失。
方法	① 线性分配（每专家等量 token）；② 匈牙利算法求解；③ 与 top-2 对比。
数据/实验	最大 110B 参数；在同等算力下优于稠密基线。
局限	O(T^3) 求解成本，只适合小 batch；无法扩展到千亿 token 级训练。

#10 奠基与路由机制 2022 • DeepMind ★ 8.0

Unified Scaling Laws for Routed Language Models

arXiv:2202.01169 • arxiv.org/abs/2202.01169

> 给出路由模型的 scaling law：参数与计算是两个独立轴，最优稀疏度随算力增长。

核心问题	路由模型的性能可以写成参数、计算、专家数的幂律函数，且存在最优稀疏度。
方法	① 参数/计算双轴 scaling；② 专家数对 loss 的影响；③ 路由算法对比（top-1/top-2/RL）。
数据/实验	最大 200B 参数；发现'更大稀疏度在固定算力下更优'但收益递减。
局限	拟合基于 2022 年前的架构；对 2026 年的细粒度+共享专家未覆盖。

#11 奠基与路由机制 2024 • 上海交大等 ★ 9.0

Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts

arXiv:2408.15664 • arxiv.org/abs/2408.15664

> 提出用偏置项（不进梯度）做负载均衡，是 DeepSeek-V3 均衡方案的原型。

核心问题	均衡目标不应干扰语言建模梯度；用非梯度的偏置更新即可维持负载。
方法	① 专家偏置 b_i；② 按负载 sign 更新；③ 与 aux loss 的对比实验。
数据/实验	最大 3B 参数、200B token；性能与均衡都优于 aux-loss 方案。
局限	偏置更新是非平稳的，gamma 需按层调；长尾负载纠正慢。

#12 奠基与路由机制 2025 · 学术界 ★ 8.0

On DeepSeekMoE: Statistical Benefits of Shared Experts and Normalized Sigmoid Gating

arXiv:2505.10860 · arxiv.org/abs/2505.10860

首次从统计学习角度证明共享专家提高样本效率、归一化 sigmoid 门控优于 softmax。

核心问题	共享专家承担跨簇通用成分，sigmoid 门控避免全局竞争，二者共同提升细粒度 MoE 的效率。
方法	① 专家估计任务的收敛分析；② 共享专家的样本效率增益；③ router 饱和/变化率/利用率的实证。
数据/实验	证明密度估计以 n^-1/2 速率收敛；实验覆盖合成数据与真实(V)L 任务。
局限	理论假设较强（簇结构、非线性）；对超大规模（>100B）未验证。

训练系统与并行（6 篇）

#13 训练系统与并行 2022 · Stanford ★ 8.0

MegaBlocks: Efficient Sparse Training with Mixture-of-Experts

arXiv:2211.15841 · arxiv.org/abs/2211.15841

用块稀疏算子重写 MoE 计算，做到'不丢 token 也不 padding'。

核心问题	MoE 的动态性可以用块稀疏矩阵乘表达，从而映射到现代 GPU 而不牺牲质量。
方法	① 块稀疏公式化；② 专用 GPU kernel（dropless）；③ 端到端训练加速。
数据/实验	端到端训练加速最高 40%（相对 Tutel/FasterMoE 基线）；不丢 token。
局限	块稀疏 kernel 的实现复杂度高；对推理侧的低延迟场景不适用。

#14 训练系统与并行 2022 · Microsoft ★ 8.0

Tutel: Adaptive Mixture-of-Experts at Scale

arXiv:2206.03382 · arxiv.org/abs/2206.03382

用自适应并行/流水线应对 MoE 的动态负载，是早期最完整的 MoE 系统之一。

核心问题	静态并行策略无法匹配 MoE 的动态负载；需要运行时自适应调整。
方法	① 自适应并行（根据负载切换 EP/DP）；② 自适应流水线；③ 分层 all-to-all。
数据/实验	支持万亿参数；相比基线最高 4.96× 加速。
局限	自适应切换有开销；对现代 NVLink/IB 拓扑的适配有限。

#15 训练系统与并行 2025 · 学术界 ★ 7.0

Scaling Laws for Upcycling Mixture-of-Experts Language Models

arXiv:2502.03009 · arxiv.org/abs/2502.03009

给出 upcycling 的 scaling law：稠密模型与 upcycled MoE 之间存在交互项。

核心问题	upcycling 的收益不是单调的：稠密模型越大收益越小，数据越多越划算。
方法	① 数据集大小/模型配置的幂律拟合；② 稠密-upcycled 交互项；③ 与从零训练的对比。
数据/实验	发现'中等稠密 + 大量新数据'是最优 upcycling 场景。
局限	实验规模有限（未到百 B 级）；交互项的物理解释不足。

#16 训练系统与并行 2026 • ByteDance/Tsinghua ★ 8.0

UniEP: Unified Expert-Parallel MoE MegaKernel for LLM Training

arXiv:2604.19241 • arxiv.org/abs/2604.19241

> 把 EP 的通信与计算融合进 MegaKernel，统一多种 EP 优化策略。

核心问题	EP 优化的碎片化（压缩/重叠/重排各自为战）可以用统一抽象 + mega-kernel 解决。
方法	① 通信计算融合的 mega-kernel；② 统一抽象适配不同 EP 配置；③ 数值稳定性保障。
数据/实验	在 Megatron-LM 生产框架中落地；支持多种 EP 优化组合。
局限	训练侧专用；推理侧的动态 batch 场景需额外适配。

#17 训练系统与并行 2026 • SJTU ★ 8.0

FEPLB: Exploiting Copy Engines for Nearly Free MoE Load Balancing in Distributed Training

arXiv:2604.19654 • arxiv.org/abs/2604.19654

> 利用 Hopper NVLink Copy Engine 不占 SM 的特性，做'近零成本'的专家重排。

核心问题	负载均衡的重排数据搬运可以完全在后台 copy engine 上完成，不抢计算资源。
方法	① Copy Engine 搬运专家；② 细粒度 per-micro-batch 均衡；③ 与 DeepEP 的对比。
数据/实验	在 Hopper 上实现近零成本重排；相比 DeepEP 显著降低均衡开销。
局限	依赖 Hopper 架构；对跨节点场景的收益有限。

#18 训练系统与并行 2026 • ZJU/PKU/SJTU ★ 7.0

Cobalt: Leveraging Expert Co-Activation for Efficient Distributed MoE Training

arXiv:2609.36959 • arxiv.org/abs/2609.36959

> 利用'专家共激活'模式（同一 token 常同时激活某些专家）优化 EP 通信与负载。

核心问题	专家不是独立被选择的，存在稳定的共激活模式；利用它可以减少跨节点传输、均衡负载。
方法	① 共激活统计分析；② 按共激活聚类放置专家；③ 通信与负载联合优化。
数据/实验	在跨节点带宽受限场景显著优于按单专家统计的基线。
局限	共激活模式随数据漂移；需要周期性重算。

模型架构代际（16 篇）

#19 模型架构代际 2024 • DeepSeek ★ 9.0

DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

arXiv:2401.06066 • arxiv.org/abs/2401.06066

> 提出细粒度专家切分 + 共享专家隔离，是 DeepSeek 系列 MoE 的起点。

核心问题	要让专家真正专业化，必须'切小'（更多更窄的专家）并'隔离通用知识'（共享专家）。
方法	① 细粒度专家（64 routed + 2 shared）；② 共享专家隔离；③ 负载均衡 + device-limited routing 前身。
数据/实验	16.4B 总参数/2.8B 激活；2T token；同等激活下优于 GShard 式粗粒度 MoE。

局限	专家切太细会导致单专家 GEMM 过小；共享专家比例需调。

#20 模型架构代际 2024 · DeepSeek ★ 9.0

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model

arXiv:2405.04434 · arxiv.org/abs/2405.04434

> MLA + DeepSeekMoE 的组合，首次证明 MoE 可以'又强又便宜'。

核心问题	MLA 压 KV、MoE 压算力，两者正交，组合后成本大幅下降。
方法	① MLA（KV 压成 latent，压缩 512 维）；② 160 routed + 2 shared、top-6；③ device-limited routing。
数据/实验	236B/21B；128K 上下文；8.1T token；相比 DeepSeek 67B 训练成本 −42.5%、KV cache −93.3%、吞吐 5.76×。
局限	MLA 的实现复杂；device-limited routing 牺牲部分路由自由度。

#21 模型架构代际 2024 · DeepSeek ★ 10.0

DeepSeek-V3 Technical Report

arXiv:2412.19437 · arxiv.org/abs/2412.19437

> MoE 训练的里程碑：aux-loss-free、MTP、FP8、DualPipe，14.8T token 零不可恢复 spike。

核心问题	算法-框架-硬件协同设计可以把 671B MoE 的训练成本压到 2.788M H800 hours。
方法	① aux-loss-free 均衡；② FP8 细粒度量化（1×128 tile / 128×128 block）；③ DualPipe 双向流水线 + 通信重叠；④ MTP 深度 1。
数据/实验	671B/37B；61 层/hidden 7168；256+1 专家/top-8/专家中间维 2048；14.8T token；2.788M H800 hours。
局限	FP8 配方对硬件有要求；aux-loss-free 的偏置漂移需监控；MTP 增加训练复杂度。

#22 模型架构代际 2025 · DeepSeek ★ 8.0

DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models

arXiv:2512.02556 · arxiv.org/abs/2512.02556

> V3 主干 + DSA 稀疏注意力 + 规模化 RL，证明 MoE 主干可与稀疏注意力解耦升级。

核心问题	长上下文效率的提升不必重训整个模型：把注意力换成 DSA 即可。
方法	① DSA（轻量 indexer 选 top-k KV 块）；② 可扩展 RL 框架；③ 大规模 agentic 数据合成。
数据/实验	AIME 2025 96.0、HMMT 94.6、Codeforces 2701（Speciale）；IMO/IOI 金牌水平。
局限	DSA 的 indexer 训练需要额外数据；RL 阶段的路由漂移是新问题。

#23 模型架构代际 2026 · DeepSeek ★ 10.0

DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

arXiv:2606.19348 · arxiv.org/abs/2606.19348

> 1M 上下文 + CSA/HCA 混合注意力 + mHC + Muon，1M 下单 token FLOPs 仅 V3.2 的 27%。

核心问题	长上下文效率的关键是'压缩 + 稀疏'的混合注意力，而不是单纯拉长 RoPE。
方法	① CSA（压缩 4× + top-512/1024 稀疏选择）；② HCA（压缩 128×）；③ mHC（流形约束超连接）；④ Muon；⑤ 前 3 层 Hash routing。
数据/实验	V4-Pro 1.6T/49B（61 层/384+1 专家/top-6/专家维 3072）；V4-Flash 284B/13B（43 层/256+1/top-6/专家维 2048）；32T token；1M 上下文 KV 仅 V3.2 的 10%。

局限	架构复杂度高；mHC 的 Sinkhorn 迭代（20 次）有开销；Hash routing 只用于前 3 层。

#24 模型架构代际 2026 • DeepSeek ★ 9.0

DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression

arXiv:2609.19969 • arxiv.org/abs/2609.19969

CED 架构 + CSA2 跨层 KV 复用 + FP4 KV，把每 token 全局 KV 压到 890 字节。

核心问题	长上下文推理的成本瓶颈是 KV cache，不是权重；KV 压缩可以做到 1/4 甚至 1/8。
方法	① CED（decoder global KV 从 encoder final hidden 投影）；② prefill 8B / decode 16B 激活；③ CSA2 跨层复用 + FP4 KV；④ SWA Bounded Replay。
数据/实验	552B 主干；45T 多模态 token；890 B/token 全局 KV（V4-Flash 的 1/4）；持久 KV 为 1/8。
局限	CED 的训练目标与常规 decoder 不同；FP4 KV 的精度损失需评估；SWA replay 是近似重建。

#25 模型架构代际 2023 • Mistral AI ★ 9.0

Mixtral of Experts

arXiv:2401.04088 • arxiv.org/abs/2401.04088

8 专家 top-2 的开源 MoE，是 MoE 进入开源生态的拐点。

核心问题	简单的 8 专家 top-2 就能在开源社区复现'大模型能力 + 小推理成本'。
方法	① 8 专家/top-2；② 32K 上下文；③ 与 Llama 2 70B/GPT-3.5 对比。
数据/实验	46.7B 总参数/12.9B 激活；32K 上下文；全面超过 Llama 2 70B。
局限	专家数少（8），组合空间有限；没有细粒度/共享专家设计。

#26 模型架构代际 2024 • Allen AI ★ 8.0

OLMoE: Open Mixture-of-Experts Language Models

arXiv:2409.02060 • arxiv.org/abs/2409.02060

全开源（权重/数据/代码/日志）的 7B/1B MoE，是研究 MoE 训练的最佳样本。

核心问题	把 MoE 的每一个细节都公开，让社区可以复现和分析路由行为。
方法	① 64 专家/top-8；② 5T token 预训练；③ 路由专业化分析（高专业化）。
数据/实验	6.9B 总参数/1.3B 激活；16 层；5T token；超过 Llama2-13B-Chat 与 DeepSeekMoE-16B。
局限	规模较小（7B），结论对千 B 级模型的外推需谨慎。

#27 模型架构代际 2025 • Alibaba Qwen ★ 9.0

Qwen3 Technical Report

arXiv:2505.09388 • arxiv.org/abs/2505.09388

Qwen3 系列：0.6-235B、dense+MoE、thinking/non-thinking 统一、thinking budget。

核心问题	把'思考模式'和'非思考模式'统一到一个模型，用 chat template 切换，避免部署两套模型。
方法	① thinking/non-thinking 统一；② thinking budget；③ 强到弱蒸馏；④ MoE 235B-A22B（128 专家/top-8/94 层/36T token）。
数据/实验	235B/22B；36T token；128 专家/top-8；94 层。

局限	thinking budget 的粒度较粗；MoE 版本的推理成本仍高。

#28 模型架构代际 2026 · Alibaba Qwen ★ 9.0

On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability

arXiv:2608.30320 · arxiv.org/abs/2608.30320

125B/6B 的超稀疏 MoE，用 GDN+QSA 混合注意力 + n-gram 外置，训练 FLOPs 仅为前代 1/9。

核心问题	容量可以放在'主干之外'（n-gram 表外置），主干只保留高效稀疏计算。
方法	① GDN + 全局注意力（每 4 层 1 个全注意力）；② QSA 稀疏注意力；③ Gated Residual（4 分支）；④ 51B n-gram 表外置 host memory；⑤ Muon。
数据/实验	125B/6B；+51B n-gram 外置；14 个预训练基准中 8 个超过 397B-A17B 前代；训练 FLOPs 约 1/9。
局限	n-gram 外置依赖 host 带宽；QSA 的 indexer 增加复杂度；小激活参数可能限制上限。

#29 模型架构代际 2025 · Moonshot ★ 9.0

Kimi K2: Open Agentic Intelligence

arXiv:2507.20534 · arxiv.org/abs/2507.20534

1.04T/32.6B、384 专家/top-8，用 MuonClip 做到 15.5T token 零 loss spike。

核心问题	Muon 的 token 效率 + QK-clip 的稳定性，可以在万亿参数上做到零 spike。
方法	① MuonClip（QK-clip）；② 384 专家/top-8；③ 大规模 agentic 数据合成 + 联合 RL。
数据/实验	1.04T/32.6B；15.5T token；SWE-Bench Verified 65.8、Tau2-Bench 66.1。
局限	QK-clip 的阈值需调；1T 模型的部署成本极高。

#30 模型架构代际 2026 · Moonshot ★ 10.0

Kimi K3: Open Frontier Intelligence

arXiv:2607.24653 · arxiv.org/abs/2607.24653

2.8T/104B、896 专家/top-16、1M 上下文、KDA，scaling efficiency 比 K2 提升 2.5×。

核心问题	极致稀疏（896 专家）+ 线性注意力（KDA）+ 更好的数据，三者叠加带来效率跃升。
方法	① KDA（Kimi Delta Attention）；② Attention Residuals；③ Stable LatentMoE（896 路由专家/top-16）；④ 百万 token agentic RL。
数据/实验	2.8T/104B；896 专家/top-16；1M 上下文；原生视觉；2.5× scaling efficiency。
局限	2.8T 权重（BF16 约 5.6TB）部署门槛极高；top-16 的通信量更大。

#31 模型架构代际 2026 · Zhipu/Tsinghua ★ 9.0

GLM-5: from Vibe Coding to Agentic Engineering

arXiv:2602.15763 · arxiv.org/abs/2602.15763

744B/40B、256 专家/80 层、DSA 稀疏注意力、异步 RL，是 GLM-4.5 的两倍规模。

核心问题	用 DSA 降低长上下文成本，用异步 RL 提升后训练效率。
方法	① DSA 稀疏注意力；② 256 专家/80 层（减少层数以降 EP 通信）；③ 异步 RL（generation/training 解耦）；④ 28.5T token。
数据/实验	744B/40B；256 专家；80 层；28.5T token；SWE-bench Verified 77.8、Terminal-Bench 2.0 56.2。

局限	层数减少可能影响深度表达；异步 RL 的一致性需保障。

#32 模型架构代际 2026 • MiniMax ★ 8.0

The MiniMax-M2 Series: Mini Activations Unleashing Max Real-World Intelligence

arXiv:2605.26494 • arxiv.org/abs/2605.26494

> 229.9B/9.8B 的'小激活'MoE，端到端为 agentic 部署设计。

核心问题	小激活参数 + agent 原生 RL，可以在 agentic 任务上达到前沿水平。
方法	① agent 数据管线（可验证轨迹）；② Forge agent-native RL（windowed-FIFO + prefix-tree 合并）；③ M2.7 自演化（自主调试训练）。
数据/实验	229.9B/9.8B；SWE Bench Pro 56.2、GDPval-AA 50、Multi-SWE 52.7。
局限	小激活可能限制知识容量；自演化机制的可控性需评估。

#33 模型架构代际 2025 • StepFun ★ 8.0

Step-3 is Large yet Affordable: Model-system Co-design for Cost-effective Decoding

arXiv:2507.19427 • arxiv.org/abs/2507.19427

> 321B/38B VLM，用 MFA 注意力 + AFD（Attention-FFN 分离）把解码成本压到最低。

核心问题	解码成本的最小化需要'模型架构'与'系统部署'联合设计，而不是各自优化。
方法	① MFA（64 query heads 共享 KV，query 降维 7168→2048）；② AFD（attention/FFN 分离）；③ 61 层/hidden 7168。
数据/实验	321B/38B；61 层；MFA 把 KV cache 大幅压缩；AFD 相比 EP 显著降低解码成本。
局限	AFD 的 token 传输开销需精细控制；MFA 的表达力需验证。

#34 模型架构代际 2025 • Meituan ★ 8.0

LongCat-Flash Technical Report

arXiv:2509.01322 • arxiv.org/abs/2509.01322

> 560B MoE，用 Zero-computation Experts 做动态计算预算（18.6-31.3B/平均 27B）。

核心问题	不同 token 的难度不同，应该用不同的计算预算，而不是固定 top-k。
方法	① Zero-computation Experts（动态预算）；② Shortcut-connected MoE（扩大重叠窗口）；③ MLA（KV 压缩 512/query 1536）。
数据/实验	560B 总参数；激活 18.6-31.3B（平均 27B）；专家中间维 2048；dense FFN 12288。
局限	动态预算的难度估计器不完美；zero-computation 专家的训练信号弱。

推理系统与部署（12 篇）

#35 推理系统与部署 2025 • DeepSeek ★ 9.0

Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures

arXiv:2505.09343 • arxiv.org/abs/2505.09343

ISCA'25 工业 track：从 V3 的工程实践反推对 AI 硬件的需求。

核心问题	MoE 的瓶颈不是算力，而是 HBM 容量、互联带宽、小 GEMM 效率；芯片设计应围绕这三点。
方法	① 分析 MLA/MoE 的硬件需求；② 给出 HBM/互联/计算单元的建议；③ 2048 H800 的实测数据。
数据/实验	2048 H800 训练；指出 HBM 容量增长应快于带宽、互联需支持多对多、计算单元需支持小 GEMM。
局限	厂商视角，部分建议与商业路线不完全一致。

#36 推理系统与部署 2024 · ByteDance/PKU ★ 9.0

MegaScale-Infer: Serving Mixture-of-Experts at Scale with Disaggregated Expert Parallelism

arXiv:2504.02263 · arxiv.org/abs/2504.02263

把 attention 和 FFN 分离到不同 GPU 组，用 ping-pong pipeline 隐藏通信。

核心问题	MoE 的 attention 和 FFN 资源需求不同，分离部署 + 独立并行策略可以最大化利用率。
方法	① attention/FFN 分离；② ping-pong pipeline（micro-batch 在两组间穿梭）；③ M2N 通信库（消除 GPU-CPU 拷贝）。
数据/实验	相比基线显著提升吞吐；M2N 降低 token dispatch 开销。
局限	分离增加 token 传输；两组 GPU 的比例需精细配置。

#37 推理系统与部署 2025 · HKUST(GZ)/HIT ★ 8.0

Efficient MoE Inference with Fine-Grained Scheduling of Disaggregated Expert Parallelism

arXiv:2512.21487 · arxiv.org/abs/2512.21487

针对 DEP 的细粒度任务调度，支持 shared experts，最大化任务重叠。

核心问题	DEP 的瓶颈是任务调度粒度太粗；把计算/通信拆成小任务并重叠可以显著提升吞吐。
方法	① 细粒度任务切分（AG/EG 两组）；② 任务调度优化问题；③ 支持 shared experts。
数据/实验	相比现有 DEP 显著提升吞吐；支持带共享专家的现代 MoE。
局限	调度开销随任务数增加；对动态 batch 的适应性需验证。

#38 推理系统与部署 2025 · CUHK-SZ/TeleAI ★ 9.0

JANUS: Disaggregating Attention and Experts for Scalable MoE Inference

arXiv:2512.13525 · arxiv.org/abs/2512.13525

attention/MoE 分离到不同 worker pool + 微秒级 activation scheduler + SLO-aware scaling，4.7× 单卡吞吐。

核心问题	分离后需要'按层激活专家数'做细粒度调度，才能平衡两个 pool 的负载。
方法	① attention/MoE 分离；② 两阶段通信；③ 微秒级 activation scheduler；④ SLO-aware 资源 scaling + 专家放置。
数据/实验	per-GPU 吞吐最高 4.7×；满足 token 级延迟 SLO。
局限	activation scheduler 的预测误差会影响效果；SLO-aware scaling 需要准确的负载模型。

#39 推理系统与部署 2026 · 工业界 ★ 8.0

PROBE: Co-Balancing Computation and Communication in MoE Inference via Real-Time Predictive Prefetching

arXiv:2602.00509 · arxiv.org/abs/2602.00509

用 gate 初始化的预测器预判下一层专家激活，提前预取并平衡计算/通信。

核心问题	MoE 推理的'双罚'（计算偏斜 + 网络拥塞）可以用预测式预取同时缓解。
方法	① Gate-Initialized Lookahead Predictor；② 硬件感知的均衡规划；③ Phase-Locked Co-Scheduling（分阶段传输）。
数据/实验	实时预测下一层专家激活；在 hiding-window 约束下优化复制与分配。
局限	预测误差会浪费带宽；控制开销需完全移出关键路径。

#40 推理系统与部署 2026 · SEU/CUHK ★ 8.0

MixServe: An Automatic Distributed Serving System for MoE Models with Hybrid Parallelism

arXiv:2601.08800 · arxiv.org/abs/2601.08800

TP-EP 混合并行 + fused AR-A2A 通信，自动选择并行配置。

核心问题	TP 的跨节点效率低、EP 的负载不均，混合并行可以取长补短。
方法	① TP-EP 混合；② fused AR-A2A；③ 自动配置搜索。
数据/实验	相比纯 TP/纯 EP 显著提升吞吐；自动搜索降低调参成本。
局限	混合并行的实现复杂度高；配置搜索需要 profiling 数据。

#41 推理系统与部署 2026 · Salesforce ★ 8.0

Least-Loaded Expert Parallelism: Load Balancing an Imbalanced Mixture-of-Experts

arXiv:2601.17111 · arxiv.org/abs/2601.17111

承认负载不均是自然的，用'最少负载'重路由 excess token 与专家参数，5× 加速。

核心问题	均衡的目标不是'完全均匀'，而是'让所有设备在最小集合延迟内完成'。
方法	① 动态重路由 excess token；② 专家参数迁移；③ 内存约束下的最小化 collective latency。
数据/实验	最高 5× 加速、4× 峰值显存下降。
局限	参数迁移的开销需控制；对超大规模 EP 的扩展性需验证。

#42 推理系统与部署 2024 · Stevens/Rice ★ 7.0

Taming Latency-Memory Trade-Off in MoE-Based LLM Serving via Fine-Grained Expert Offloading

arXiv:2502.05370 · arxiv.org/abs/2502.05370

从专家选择模式和 prompt 语义提取预取提示，做细粒度 offloading。

核心问题	offloading 的关键是'预取准确性'；专家选择模式 + 语义提示可以提高命中率。
方法	① 细粒度专家选择模式提取；② prompt 语义提示；③ prefetch/cache/offload 联合。
数据/实验	低延迟 + 内存高效；相比粗粒度 offloading 显著改善。
局限	语义提示依赖模型内部状态；对分布漂移的鲁棒性需验证。

#43 推理系统与部署 2026 · Tsinghua ★ 8.0

Achieving Cloud-Grade SLOs for Local Mixture-of-Experts Inference through CPU-GPU Hybrid Design

arXiv:2606.10493 · arxiv.org/abs/2606.10493

双 RTX 5090 上做到 1800 token/s prefill、45K prompt 30 秒、云级 SLO。

核心问题	本地 MoE 推理的瓶颈是 prefill 和并发；流式加载 + CPU-GPU 混合可以逼近云端。
方法	① SLP 流式加载 prefill；② DSLP + SmallEP；③ intra-node PD 分离 + 零拷贝共享权重；④ AVX-512 FP8 GEMV。
数据/实验	prefill 1200-1800 token/s；45K prompt 30s；并发延迟增加 <15%、吞吐 +50%。
局限	依赖双路 CPU 和大内存；对更大模型（>500B）的扩展性需验证。

#44 推理系统与部署 2026 • Nanjing/Huawei ★ 8.0

UBEP: Re-architecting Expert Parallelism Communication Library for Production Superpods

arXiv:2607.06202 • arxiv.org/abs/2607.06202

面向 NVL72/576、CloudMatrix384 等超节点的 EP 通信库，解决 BSP 串行化/同步开销/负载不均。

核心问题	超节点的统一地址空间和高带宽不等于高效 EP；需要重新设计通信编排。
方法	① 打破 BSP 粗粒度同步；② 低开销同步；③ 距离感知的负载均衡调度。
数据/实验	在超节点上显著优于通用 EP 库；支持不规则 token 流量。
局限	依赖超节点硬件；对普通 IB 集群的收益有限。

#45 推理系统与部署 2026 • NVIDIA ★ 9.0

NCCL EP: Towards a Unified Expert Parallel Communication API for NCCL

arXiv:2603.13606 • arxiv.org/abs/2603.13606

NVIDIA 官方 EP 通信库：ncclEpDispatch/ncclEpCombine，LL/HT 双模式。

核心问题	EP 通信需要标准化 API，让上层框架不必自己造轮子；LL 模式服务 decode，HT 模式服务训练/prefill。
方法	① 基于 NCCL Device API；② LL 模式（1-128 token decode）；③ HT 模式（训练/prefill）；④ C/Python 接口。
数据/实验	统一 API；LL/HT 双模式覆盖不同 batch 规模。
局限	生态绑定 NVIDIA；对非 NCCL 后端的适配需额外工作。

#46 推理系统与部署 2026 • Fudan/SJTU/Shanghai AI Lab ★ 7.0

NanoCP: Taming Load Imbalance in Data-Expert Parallel Decoding under Variable-Length Workloads

arXiv:2605.21100 • arxiv.org/abs/2605.21100

解决 data-expert 混合并行在变长负载下的解码不均衡。

核心问题	DP 复制 attention、EP 切分专家时，变长请求会导致两组负载严重不匹配。
方法	① 变长负载建模；② data-expert 并行调度；③ 解码阶段的负载均衡。
数据/实验	在变长 workload 下显著改善吞吐/延迟。
局限	调度模型依赖请求长度分布；对突发流量的适应性需验证。

量化与压缩（4 篇）

#47 量化与压缩 2023 • Microsoft ★ 8.0

Mixture of Quantized Experts (MoQE): Complementary Effect of Low-bit Quantization and Robustness

arXiv:2310.02410 • arxiv.org/abs/2310.02410

只量化专家权重到 2-bit，发现专家层比普通 FFN 更鲁棒。

核心问题	MoE 的专家层对量化更鲁棒，可以承受比 dense FFN 更低的比特。
方法	① 仅专家权重量化（2-bit）；② 与 dense FFN 量化的对比；③ 内存/延迟收益。
数据/实验	专家权重 2-bit；内存大幅下降；性能损失小（多数情况无需额外训练）。
局限	只量化权重（weight-only）；激活量化未覆盖；2-bit 的实际 kernel 支持有限。

#48 量化与压缩 2023 • IST Austria ★ 8.0

QMoE: Practical Sub-1-Bit Compression of Trillion-Parameter Models

arXiv:2310.16795 • arxiv.org/abs/2310.16795

把 1.6T SwitchTransformer 压到 <160GB（0.8 bit/param，20×），单 GPU 一天完成。

核心问题	万亿 MoE 的显存问题可以用'子 1-bit 压缩 + 定制解码 kernel'解决。
方法	① 可扩展压缩算法；② 定制 GPU 解码 kernel；③ 端到端压缩推理。
数据/实验	1.6T → <160GB（0.8 bit/param，20×）；单 GPU 一天内完成；精度损失小。
局限	压缩后的 kernel 吞吐低于原生；对现代 MoE 架构的适配需重做。

#49 量化与压缩 2025 • UConn 等 ★ 7.0

Dynamic Expert Quantization for Scalable Mixture-of-Experts Inference

arXiv:2511.15015 • arxiv.org/abs/2511.15015

把单 GPU MoE 的精度分配建模为在线预算约束问题，按专家热度动态调比特。

核心问题	专家利用率是重尾的、热集合会漂移；固定比特分配不是最优。
方法	① 运行时精度分配；② 硬显存约束下的在线优化；③ 热专家高精度/冷专家低精度。
数据/实验	单 GPU 硬显存约束下提升吞吐；适应工作负载漂移。
局限	在线分配有决策开销；精度切换的 kernel 支持复杂。

#50 量化与压缩 2026 • Amazon ★ 8.0

MonoMoE: An Efficient Fused Mega-kernel for Quantized MoE Decoding

arXiv:2609.04244 • arxiv.org/abs/2609.04244

weight-major persistent megakernel，把整个 decode step 融合成一个 kernel。

核心问题	token-major grouped GEMM 在小 batch 下 padding 严重；weight-major 可以让专家权重流主导。
方法	① weight-major 布局；② persistent grid；③ 融合 routing/top-k/量化/GEMM/激活/reduction；④ warp specialization。
数据/实验	集成 vLLM；block-wise 量化 decode 吞吐提升。

局限	实现复杂度极高；对 prefill（大 batch）的收益有限。

多模态与注意力（3 篇）

#51 多模态与注意力 2024 • DeepSeek ★ 8.0

DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding

arXiv:2412.10302 • arxiv.org/abs/2412.10302

> 动态 tiling 视觉编码 + DeepSeekMoE + MLA 的多模态 MoE。

核心问题	高分辨率图像需要动态 tiling；MoE 主干 + MLA 可以兼顾容量与推理效率。
方法	① 动态 tiling；② DeepSeekMoE + MLA；③ 视觉-语言数据。
数据/实验	在 VQA/OCR/文档/表格/图表理解上表现强；MLA 降低 KV。
局限	视觉 token 数量大，放大专家负载不均；多模态 RL 未覆盖。

#52 多模态与注意力 2026 • SJTU ★ 7.0

VisMMOE: Exploiting Visual-Expert Affinity for Efficient Visual-Language MoE Offloading

arXiv:2605.05899 • arxiv.org/abs/2605.05899

> 视觉 token 剪枝不仅省计算，还让专家访问更集中、更可预测，提升 offloading 命中率。

核心问题	视觉 token 与专家的亲和性可以被利用：剪枝后专家工作集更小、更稳定。
方法	① 亲和性感知的 token 压缩；② lookahead 专家预测；③ cache/pipeline 编排。
数据/实验	在视觉密集输入下显著改善 offloading 命中率与延迟。
局限	剪枝可能损失细粒度视觉信息；对文本为主的负载收益有限。

#53 多模态与注意力 2025 • Moonshot ★ 8.0

Kimi Linear: An Expressive, Efficient Attention Architecture

arXiv:2510.26692 • arxiv.org/abs/2510.26692

> KDA 线性注意力首次在公平对比下全面超过全注意力（短/长上下文/RL）。

核心问题	线性注意力不是'次优替代'，只要门控足够细粒度（KDA），它可以超越全注意力。
方法	① KDA（对角+低秩转移矩阵）；② chunkwise 算法；③ KDA+MLA 层间混合。
数据/实验	3B 激活/48B 总参数；相同训练配方下全面超过 MLA；长上下文更快。
局限	线性注意力的长程记忆仍是瓶颈；混合比例需调。

结构创新与均衡细节（3 篇）

#54 结构创新与均衡细节 2022 • UCLA ★ 8.0

Towards Understanding Mixture of Experts in Deep Learning

arXiv:2208.02813 • arxiv.org/abs/2208.02813

首个 MoE 理论学习：数据簇结构 + 专家非线性是 MoE 成功的两个关键条件。

核心问题	Router 学到的是簇中心特征，把复杂问题分解为多个线性子问题，每个专家解决一个。
方法	① 簇结构分类问题的形式化；② 两层非线性 CNN 专家的可学习性证明；③ 路由不会坍缩的条件。
数据/实验	证明 MoE 可以学到单专家无法学习的簇结构问题；给出路由学习簇中心的理论。
局限	假设较强（簇结构、非线性）；对真实大模型的适用性是类比而非证明。

#55 结构创新与均衡细节 2025 · 学术界 ★ 8.0

Demons in the Detail: On Implementing Load Balancing Loss for Training Specialized Mixture-of-Experts

arXiv:2501.11873 · arxiv.org/abs/2501.11873

揭示负载均衡损失的实现细节（f_i 计数口径）会让'同样超参'训出完全不同的专家分布。

核心问题	aux loss 的魔鬼在细节：是否含 padding、是否按 top-k 重复计数，会显著改变均衡行为。
方法	① 不同实现口径的对比；② 过度均衡伤害专业化；③ 推荐实现规范。
数据/实验	发现'过度均衡'会迫使专家处理非专长 token，损害性能。
局限	实验规模中等；对 aux-loss-free 的对应问题未覆盖。

#56 结构创新与均衡细节 2025 · Tsinghua 等 ★ 7.0

BigMac: A Communication-Efficient Mixture-of-Experts Model Structure for Fast Training and Inference

arXiv:2502.16927 · arxiv.org/abs/2502.16927

把 all-to-all 从高维挪到低维：CDAC 改成 DCCA，通信量大幅下降。

核心问题	细粒度 MoE 的通信瓶颈来自'通信发生在最高维'（CDAC）；在专家入口降维、出口升维，让 all-to-all 走低维即可。
方法	① 专家入口加降维投影、出口加升维投影；② 重设计小专家结构补偿降维后的容量损失；③ 在 Megatron/Tutel/DeepSpeed-Inference 上实测。
数据/实验	同等专家数与总参数下质量可比或更好；训练端到端延迟最多降 3.09×，推理吞吐最多升 3.11×。
局限	降维/升维投影引入额外参数与算子，需与框架 EP 调度配合；未在 >1T 规模验证。

附录 C · 80 篇相关工作速览

共 80 篇，按主题分组；每篇一句中文结论，便于按方向检索。标题取自 arXiv 元数据。

#	arXiv	年份	论文标题	一句话
训练/均衡/通信				
1	2510.19262	2025	RailS: Load Balancing for All-to-All Communication in Distributed Mixture-of-Experts Training	把 all-to-all 当铁路调度：按 rail 分配路径，减少拥塞与 straggler。
2	2603.28768	2026	CRAFT: Fine-Grained Cost-Aware Expert Replication For Efficient Mixture-of-Experts Serving	按'复制成本 vs 收益'细粒度决定哪些专家复制几份。
3	2608.28511	2026	Training Communication-Efficient Mixture-of-Experts Language Models with Layer Re-Configuration	动态重配层结构，把通信密集层改造成通信较轻的形态。
4	2605.05049	2026	Piper: Efficient Large-Scale MoE Training via Resource Modeling and Pipelined Hybrid Parallelism	资源建模 + 流水线混合并行，自动搜最优训练配置。
5	2602.11686	2026	LAER-MoE: Load-Adaptive Expert Re-layout for Efficient Mixture-of-Experts Training	按实时负载重排专家布局，减少训练时的跨节点流量。
6	2602.19938	2026	A Replicate-and-Quantize Strategy for Plug-and-Play Load Balancing of Sparse Mixture-of-Experts LLMs	复制 + 量化双管齐下，即插即用地均衡负载。
7	2503.05066	2025	Capacity-Aware Inference: Mitigating the Straggler Effect in Mixture of Experts	按容量感知调度，缓解 MoE 推理的尾部长尾（straggler）。
8	2610.09372	2026	Expert Coupling in MoE Pretraining: Reducing All-to-All Overhead with Correlated Placement and Token Shuffling	利用专家共激活的相关性放置专家，减少 all-to-all。
9	2511.11505	2025	FarSkip-Collective: Unhobbling Blocking Communication in Mixture of Experts Models	把阻塞通信拆成可跳过的细粒度集合操作，减少等待。
10	2609.34340	2026	VarioPath: Workload-Aware All-to-All Communication for PCIe GPU Clusters	面向 PCIe 集群的负载感知 all-to-all 路径选择。
11	2607.19539	2026	Fine-grained Computation-Communication Overlap via Tile-level Signaling and Scheduling for Mixture-of-Experts	用 tile 级信号把计算/通信重叠做到更细粒度。
12	2610.36339	2026	INC-MoE: Overlapping In-Network Collectives and Expert Computation for Expert-Parallel Mixture-of-Experts Inference	把集合通信下放到网络内，与专家计算重叠。
13	2505.13345	2025	Occult: Optimizing Collaborative Communication across Experts for Accelerated Parallel MoE Training and Inference	利用专家间的协作通信模式减少冗余传输。
14	2605.06055	2026	Relay Buffer Independent Communication over Pooled HBM for Efficient MoE Inference on Ascend	在昇腾上用池化 HBM 做中继缓冲，降低通信开销。
15	2605.06206	2026	Federation of Experts: Communication Efficient Distributed Inference for Large Language Models	联邦式专家分布：按请求路由到'专家联盟'，减少跨域通信。
16	2506.12708	2025	Serving Large Language Models on Huawei CloudMatrix384	华为 CloudMatrix384 超节点上的大模型服务实践。

#	arXiv	年份	论文标题	一句话
17	2510.25258	2025	MoEntwine: Unleashing the Potential of Wafer-scale Chips for Large-scale Expert Parallel Inference	晶圆级芯片上的大规模 EP：把专家'编织'在晶圆内。
18	2603.07006	2026	Mozart: Modularized and Efficient MoE Training on 3.5D Wafer-Scale Chiplet Architectures	3.5D 晶圆级 chiplet 上的模块化 MoE 训练。
推理系统/服务				
19	2607.18002	2026	ExpertPlex: A High-Goodput Disaggregated Serving System for MoE LLMs with Adaptive Persistent Kernels	分离式服务 + 自适应持久缓存，提升 goodput。
20	2609.00993	2026	AInfer-PD: Communication-Safe In-Place Prefill-Decode Multiplexing for Distributed MoE Rollouts	prefill/decode 原地多路复用，通信安全。
21	2510.26730	2025	ExpertFlow: Adaptive Expert Scheduling and Memory Coordination for Efficient MoE Inference	自适应专家调度 + 内存协调。
22	2410.17954	2024	ExpertFlow: Efficient Mixture-of-Experts Inference via Predictive Expert Caching and Token Scheduling	预测式专家缓存 + token 调度。
23	2609.38090	2026	Mira: Memory-Efficient MoE Inference Using Adaptive Caching and Predictive Expert Staging	自适应缓存 + 预测式专家暂存。
24	2508.21706	2025	Accelerating Mixture-of-Experts Inference by Hiding Offloading Latency with Speculative Decoding	用投机解码把专家 offloading 的延迟藏起来。
25	2505.08944	2025	Toward Cost-Efficient Serving of Mixture-of-Experts with Asynchrony	异步化服务降低 MoE 的单位请求成本。
26	2505.06481	2025	QoS-Efficient Serving of Multiple Mixture-of-Expert LLMs Using Partial Runtime Reconfiguration	多 MoE 模型共享部署，按 QoS 部分重配。
27	2605.27081	2026	ReMoE: Boosting Expert Reuse through Router Fine-Tuning in Memory-Constrained MoE LLM Inference	微调 router 提高专家复用率，缓解显存压力。
28	2607.05116	2026	Communication-Aware Placement and Pruning for Efficient Mixture-of-Experts Inference	通信感知的专家放置 + 剪枝。
29	2610.09424	2026	Democratizing MoE inference on commodity GPUs with CoMoE	在消费级 GPU 上跑 MoE 的协同推理。
30	2603.27624	2026	Expert Streaming: Accelerating Low-Batch MoE Inference via Multi-chiplet Architecture and Dynamic Expert Trajectory Scheduling	多 chiplet 架构 + 专家流式加载，加速低 batch 推理。
31	2610.26991	2026	3D-MoE: Complementarity-Guided Hardware-Software Co-Design for MoE Decode on 3D-Stacked NMP	3D 堆叠芯片上的 MoE decode 软硬协同。
32	2610.37442	2026	CRISP: Co-designing Pruning and Scheduling for Efficient MoE Inference on Edge Servers	边缘服务器上的剪枝+调度协同。
33	2601.21198	2026	ZipMoE: Efficient On-Device MoE Serving via Lossless Compression and Cache-Affinity Scheduling	无损压缩 + 缓存亲和调度，端侧 MoE 服务。
34	2604.18788	2026	Efficient Mixture-of-Experts LLM Inference with Apple Silicon NPUs	Apple Silicon NPU 上的 MoE 推理。

#	arXiv	年份	论文标题	一句话
35	2506.23635	2025	Towards Building Private LLMs: Exploring Multi-Node Expert Parallelism on Apple Silicon for Mixture-of-Experts Large Language Model	Apple Silicon 集群上的多节点 EP，做私有 LLM。
36	2610.23006	2026	Efficient CPU-GPU collaborative MoE inference via Adaptive Expert Assignment	CPU-GPU 协同，按负载自适应分配专家。
37	2504.15299	2025	D²MoE: Dual Routing and Dynamic Scheduling for Efficient On-Device MoE-based LLM Serving	端侧双路由 + 动态调度。
38	2511.22481	2025	OmniInfer: System-Wide Acceleration Techniques for Optimizing LLM Serving Throughput and Latency	系统级加速：批处理、KV、调度的综合优化。
39	2609.39223	2026	QATFactory: A Versatile, Deployment-Aligned Framework for Quantization-aware Training and Distillation of LLMs	面向部署的 QAT 框架，支持 MoE。
量化/压缩				
40	2506.07533	2025	MoQAE: Mixed-Precision Quantization for Long-Context LLM Inference via Mixture of Quantization-Aware Experts	用'量化专家'混合不同精度，长上下文推理更省。
41	2607.20981	2026	Beyond Independent Optimization: Compression, MoE Routing, and Quantization Interactions in Multimodal Edge Intelligence	压缩、路由、量化三者相互影响，必须联合优化。
42	2609.01962	2026	Post-Training Ternarization of Qwen3-4B Capability, Effective Bit Budget, Storage Compression, and Deployment	Qwen3-4B 的后训练三值化：能力/比特预算/存储压缩。
43	2610.02241	2026	Hardware-Native Joint Sparse-Quantization for Trillion-Scale Mixture-of-Experts	万亿 MoE 的硬件原生联合稀疏-量化。
44	2504.02658	2025	MiLo: Efficient Quantized MoE Inference with Mixture of Low-Rank Compensators	低秩补偿器修复量化误差。
45	2607.16184	2026	PagedWeight: Efficient MoE LLM Serving with Dynamic Quality-Aware Weight Quantization	按质量要求动态调权重量化精度，分页管理。
46	2607.15810	2026	QUADS: Stabilizing NVFP4 Reinforcement Learning for MoE via QUantization-error Alignment across Dual Sides	NVFP4 RL 的量化误差对齐，稳定 MoE 训练。
47	2608.08910	2026	Tied Trit-Planes: Constraining PTQTP to a Uniform Nine-Level Quantizer, with a Persistent Folded Format for Disk-Streamed Mixture-of-Experts Serving	把 PTQ 三值平面约束到统一九级量化器。
架构/upcycling/混合				
48	2604.19835	2026	Expert Upcycling: Shifting the Compute-Efficient Frontier of Mixture-of-Experts	按专家重要性选择性 upcycling，移动算力效率前沿。
49	2610.12599	2026	STAPLE: Static Sparse Upcycling of Pretrained Checkpoints via Embedding Modules	静态稀疏 upcycling + embedding 模块。
50	2502.00965	2025	CLIP-UP: A Simple and Efficient Mixture-of-Experts CLIP Training Recipe with Sparse Upcycling	把 CLIP 稀疏 upcycling 成 MoE，训练更省。
51	2507.18671	2025	Innovator: Scientific Continued Pretraining with Fine-grained MoE Upcycling	科学继续预训练：细粒度 MoE upcycling。

#	arXiv	年份	论文标题	一句话
52	2605.26496	2026	Dense2MoE: Pushing the Pareto Frontier of On-Device LLMs via Unified Pruning and Upcycling	剪枝+upcycling 统一，推端侧 LLM 的 Pareto 前沿。
53	2603.13364	2026	FineRMoE: Dimension Expansion for Finer-Grained Expert with Its Upcycling Approach	维度扩展做更细粒度专家 + upcycling。
54	2610.11317	2026	DivMoE: Fine-Grained MoE Upcycling via Cross-Domain Expert Composition	跨域专家组合的细粒度 upcycling。
55	2509.00679	2025	Router Upcycling: Leveraging Mixture-of-Routers in Mixture-of-Experts Upcycling	upcycling 时复制 router 而不是专家。
56	2408.08274	2024	BAM! Just Like That: Simple and Efficient Parameter Upcycling for Mixture of Experts	简单高效的参数 upcycling。
57	2504.03624	2025	Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models	Mamba-Transformer 混合家族，效率优先。
58	2610.38095	2026	Nemotron 3.5 Super and Lightning: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Models for Agentic AI	Nemotron 3.5：MoE + Mamba 混合，开放高效。
59	2604.12374	2026	Nemotron 3 Super: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning	Nemotron 3 Super：同系列中杯。
60	2606.15007	2026	Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning	Nemotron 3 Ultra：同系列大杯。
61	2408.12570	2024	Jamba-1.5: Hybrid Transformer-Mamba Models at Scale	Jamba-1.5：Transformer-Mamba 混合规模化。
62	2505.15431	2025	Hunyuan-TurboS: Advancing Large Language Models through Mamba-Transformer Synergy and Adaptive Chain-of-Thought	混元 TurboS：Mamba-Transformer 协同。
63	2610.07348	2026	Stepped MoE: Segment-Level Routing with Configurable Inference Complexity	分段路由，推理复杂度可配置。
64	2609.04575	2026	Training-Free Halving of Activated Experts in Fine-Grained Mixture-of-Experts Models	免训练把激活专家数减半。
65	2312.07035	2023	HyperRouter: Towards Efficient Training and Inference of Sparse Mixture of Experts	超网络动态生成 router 参数，训练推理更高效。
66	2506.14038	2025	Load Balancing Mixture of Experts with Similarity Preserving Routers	保持相似度的 router，均衡但不破坏专业化。
多模态/长上下文/应用				
67	2509.17765	2025	Qwen3-Omni Technical Report	全模态（文本/图/音/视频）MoE 模型。
68	2605.05225	2026	MACS: Modality-Aware Capacity Scaling for Efficient Multimodal MoE Inference	按模态调整专家容量，多模态推理更省。
69	2604.19503	2026	ReaLB: Real-Time Load Balancing for Multimodal MoE Inference	多模态 MoE 的实时负载均衡。
70	2502.11096	2025	Mixture of Tunable Experts -- Behavior Modification of DeepSeek-R1 at Inference Time	推理时调专家行为，修改 R1 的风格。
71	2603.21065	2026	LongCat-Flash-Prover: Advancing Native Formal Reasoning via Agentic Tool-Integrated Reinforcement Learning	LongCat 的形式推理/工具集成版本。

#	arXiv	年份	论文标题	一句话
72	2610.28998	2026	FlashNext-RL: A System Optimization Study for Long-Horizon Agentic RL of Qwen3.8-Flash-Next	Qwen3.8-Next 的长程 agentic RL 系统优化。
73	2502.12051	2025	How to Upscale Neural Networks with Scaling Law? A Survey and Practical Guidelines	scaling law 的综述与实操指南。
74	2605.10556	2026	EnergyLens: Interpretable Closed-Form Energy Models for Multimodal LLM Inference Serving	多模态 LLM 推理能耗的闭式模型。
75	2505.02214	2025	An Empirical Study of Qwen3 Quantization	Qwen3 量化的实证研究。
76	2609.15627	2026	DeepSeek-V4-Flash on AMD gfx90a: Correctness Recovery and Inference Performance Engineering	V4-Flash 在 AMD MI250 上的正确性恢复与性能工程。
77	2610.16890	2026	FORGE: Procedural Weight Generation for Sparse MoE Inference	程序化生成专家权重，减少存储。
78	2605.08639	2026	ReLibra: Routing-Replay-Guided Load Balancing for MoE Training in Reinforcement Learning	RL 中用路由重放引导负载均衡。
79	2610.21682	2026	Router Drift Control: Mitigating Off-Policy Degradation in Mixture-of-Experts Reinforcement Learning	控制 router 漂移，缓解 RL 的 off-policy 退化。
80	2503.08467	2025	Accelerating MoE Model Inference with Expert Sharding	专家分片加速推理。

报 告 完

本报告基于截至 2026-11-03 的公开论文与技术报告整理；模型参数、训练成本等数字均标注来源口径，
未披露的数据不做推测。arXiv 编号可直接在 arxiv.org/abs/<id> 访问原文。
报告由 ZCode 生成，正文与卡片数据全部落盘于 ai_workspace/moe-survey-20261103。